

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий
Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: Информационные системы и технологии

ОТЧЕТ

по проектной практике

Студент: Котельников Юрий Вениаминович Группа: 251-332

Место прохождения практики: Московский Политех, кафедра Информатика
и информационные технологии

Подпись:



Отчет принят с оценкой _____ Дата 20.05.2026

Руководитель практики: Семенова Валерия Валерьевна

Подпись:

Москва 2026

Оглавление

ВВЕДЕНИЕ	4
1. Общая информация о проекте	6
2. Общая характеристика деятельности организации	6
3. Описание задания по проектной практике.....	7
Базовая часть:	7
Вариативная часть: создание Telegram-бота на Python.....	8
4. Описание достигнутых результатов по проектной практике	10
ЗАКЛЮЧЕНИЕ	12
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	13
ПРИЛОЖЕНИЕ А	14
Листинг А.1 — index.html.....	14
Скриншот Б.1 — Главная страница сайта проекта SPARQ	48
Скриншот Б.2 — Главная страница сайта проекта SPARQ	48
Скриншот В.1 — Страница «О проекте»	49
Скриншот В.2 — Страница «О проекте»	49
Скриншот Г.1 — Страница «Участники», описание вклада каждого участника, выполненные задачи.....	50
Скриншот Г.2 — Страница «Участники», описание вклада каждого участника, выполненные задачи.....	50
Скриншот Г.3 — Страница «Участники», описание вклада каждого участника, выполненные задачи.....	51
Скриншот Г.4 — Страница «Участники», описание вклада каждого участника, выполненные задачи.....	51
Скриншот Д.1 — Страница «Журнал», описание этапов развития проекта.	52
Скриншот Д.2 — Страница «Журнал», описание этапов развития проекта.	53
Скриншот Д.3 — Страница «Журнал», описание этапов развития проекта.	53
Скриншот Е.1 — Страница «Ресурсы», ссылки на платформу, дизайн, тг-бота.....	54
ПРИЛОЖЕНИЕ Б.....	54
Листинг Ж.1 — bot.py	54
Листинг З.1 — config.py.....	62

Скриншот И.1 — Описание тг-бота, аватар, юзернейм.	63
Скриншот К.1 — Работа тг-бота, ответы нейросети.	64

ВВЕДЕНИЕ

Актуальность. Цифровизация образования требует новых инструментов: преподаватели тратят десятки часов на ручное создание конспектов и раздаточных материалов, студентам сложно структурировать знания из видеолекций, а командная работа учебных групп ведётся в разрозненных мессенджерах без единой системы задач. Московский Политехнический университет насчитывает более 20 000 студентов — создание единой цифровой среды для учёбы и проектной деятельности является актуальной задачей.

Цель практики — закрепление и углубление теоретических знаний в области разработки IT-продуктов, а также формирование практических навыков полного цикла разработки: проектирование, реализация, тестирование, деплой и документирование.

Задачи практики:

1. Создать и наполнить контентом Telegram-канал и VK-сообщество.
2. Настроить Git-репозиторий и освоить базовые команды работы с системой контроля версий.
3. Оформить документацию проекта в формате Markdown.
4. Создать статический веб-сайт проекта с использованием HTML и CSS.
5. Разработать Telegram-бота на языке Python для навигации по проекту.
6. Подготовить итоговый отчёт по практике.

Объект практики — процесс разработки AI-платформы SPARQ для сопровождения учебного процесса и командной работы в Московском Политехническом университете.

Предмет практики — методы и инструменты создания веб-приложений, REST API, ML-сервисов и командных инструментов (Task Manager, Messenger) для студенческой аудитории.

Период прохождения практики — 02.02.2026 – 23.05.2026.

Место прохождения практики — Московский Политех, кафедра «Информатика и информационные технологии», проект «SPARQ»

Практическая значимость — разработанные инструменты (сервис расшифровки видео, Task Manager, Messenger, сайт проекта) могут быть использованы непосредственно в учебном процессе Мосполитеха для автоматизации работы с лекционными материалами и организации командной проектной деятельности.

Личный вклад автора отчёта заключается в:

- Логировании и просмотре ошибок на стенде
- шаблонах env для новых сервисов, шаблоне PR и чеклист ревью
- иконках для empty state
- многочисленных исследованиях продуктов и интеграций, потенциально полезных для проекта.

1. Общая информация о проекте

Название проекта. «SPARQ» — AI-платформа для студентов и преподавателей Московского Политехнического университета.

Цели и задачи проекта. Цель — создать единую цифровую среду для учебного процесса в Мосполитехе: автоматическая расшифровка видеолекций, управление задачами, командная коммуникация. Задачи: разработать ML-пайплайн speech-to-text, REST API, фронтенд, модуль Task Manager, модуль Messenger, задеплоить и задокументировать платформу.

Технологический стек. Frontend: React, TypeScript, Vite. Backend: Python, FastAPI, SQLAlchemy, Alembic. База данных: PostgreSQL. Файловое хранилище: S3 (twcstorage.ru). ML-сервис: Python, Whisper STT, Sber STT, PyTorch. DevOps: Docker, Coolify, GitHub Actions. Дизайн: Figma.

2. Общая характеристика деятельности организации

Наименование заказчика. Московский Политехнический университет (внутренний проект).

Организационная структура. Проект ведётся студентами кафедры «Информатика и информационные технологии» под руководством кураторов Осьмина Владимира Вячеславовича и Бунакова Владислава Евгеньевича. Команда насчитывает 30+ участников: бэкенд- и фронтенд-разработчики, ML-инженеры, дизайнеры, аналитики, маркетологи.

Описание деятельности. Университет ежегодно производит сотни часов видеолекций, однако их текстовое сопровождение и структурирование остаётся ручным. Параллельно студенческие проектные команды используют сторонние инструменты (Telegram, Notion, Trello), что создаёт

разрозненность данных. Проект SPARQ решает обе проблемы: автоматизирует создание текстовых материалов из видео и предоставляет единую платформу для командной работы.

3. Описание задания по проектной практике

Задание включало базовую и вариативную части.

Базовая часть:

Настройка Git и репозитория. Создан репозиторий на GitHub, освоены базовые команды (clone, commit, push, branch). Регулярно фиксировались изменения с осмысленными сообщениями.

Написание документов в Markdown. Оформлены описание проекта, журнал прогресса, техническая документация.

Создание статического веб-сайта. Сайт создан с использованием HTML и CSS. Включены страницы:

- Главная страница с аннотацией
- «О проекте» с описанием
- «Участники» с личным вкладом
- «Журнал»
- «Ресурсы» со ссылками

Сайт оформлен графическими материалами, реализована тёмная тема в фирменных цветах проекта.

Взаимодействие с организацией-партнёром. Проект SPARQ разрабатывается в интересах Московского Политехнического университета. В ходе практики проводились рабочие встречи с кураторами, а так же была организована экскурсия в офис компании «Sila Union».

Отчёт по практике. Составлен отчёт в форматах DOCX и PDF, загружен в репозиторий (папка reports/) и в СДО.

Вариативная часть: создание Telegram-бота на Python

В рамках вариативной части практики был разработан Telegram-бот на языке Python с использованием библиотеки aiogram 3 и интеграцией нейросети через OpenRouter API. Бот реализует сценарий диалогового взаимодействия с искусственным интеллектом и предназначен для демонстрации возможностей AI-ассистента в рамках проекта SPARQ. Листинг кода приведён в Приложении к отчёту.

Основные возможности бота;

Приветствие и главное меню. При команде /start бот выводит персонализированное приветственное сообщение и интерактивную reply-клавиатуру с кнопками:

- «Новый диалог»
- «Системный промпт»
- «Статистика»
- «Помощь»

Диалог с нейросетью. Основная функция бота — общение с языковой моделью. Пользователь отправляет текстовое сообщение, бот передаёт его в OpenRouter API и возвращает ответ от выбранной нейросети (по умолчанию — deepseek/deepseek-chat).

Поддерживается переключение на любые модели платформы: GPT-4, Claude, Gemini, Llama и др. Во время генерации ответа отображается индикатор «печатает».

Память контекста. Бот сохраняет историю диалога каждого пользователя (до 10 пар «вопрос-ответ») и передаёт её в каждый новый запрос. Благодаря этому модель помнит предыдущие сообщения и поддерживает связный разговор. История хранится в памяти процесса и индивидуальна для каждого пользователя

Раздел «Новый диалог». Команда `/new` или соответствующая кнопка полностью очищает историю текущего диалога. Это позволяет начать разговор с чистого листа, не перезапуская бота.

Раздел «Системный промпт». Команда `/system` позволяет пользователю задать собственную инструкцию для нейросети (роль, стиль ответов, тематика). После ввода нового промпта история диалога автоматически сбрасывается, чтобы модель сразу применила новые правила поведения. Реализована возможность отмены изменения.

Раздел «Статистика». Команда `/stats` выводит информацию о текущей сессии: общее количество сообщений, число запросов пользователя, число ответов ИИ, а также текущий системный промпт.

Раздел «Помощь». Команда `/help` отображает справку по всем доступным командам и кратко объясняет, что такое системный промпт и как пользоваться ботом.

Технические детали:

- Язык и библиотеки: Python 3.11, aiogram 3, openai (SDK), python-dotenv
- Подключение к нейросети: OpenRouter API (через OpenAI-совместимый клиент)
- Архитектура: асинхронная обработка через `asyncio`, FSM (Finite State Machine) для управления состояниями пользователя

- Хранилище состояний: MemoryStorage от aiogram
- Обработчики: CommandHandler для команд /start, /new, /system, /stats, /help; MessageHandler для обработки текстовых сообщений и фильтрации по состояниям FSM
- Конфигурация: токены и API-ключи вынесены в файл .env и загружаются через python-dotenv
- Токен бота: получен через @BotFather, ключ нейросети — через личный кабинет на openrouter.ai
- Логирование: стандартный модуль logging с выводом ошибок API и действий пользователей
- Обработка ошибок: при сбое запроса к нейросети последнее сообщение пользователя удаляется из истории, чтобы не нарушить контекст; пользователю выводится понятное сообщение об ошибке
- Длинные ответы: сообщения свыше 4096 символов автоматически разбиваются на части (ограничение Telegram API)

Текущий статус. На момент сдачи отчёта код бота полностью написан, все функции реализованы и протестированы локально. Проведена проверка: команды отрабатывают корректно, запросы к OpenRouter API проходят успешно, контекст диалога сохраняется между сообщениями, переключение системного промпта работает, статистика выводится без ошибок. Бот не развёрнут на удалённом сервере, но готов к деплою (например, через Docker на VPS или Coolify). Исходный код загружен в репозиторий GitHub.

4. Описание достигнутых результатов по проектной практике

В ходе практики достигнуты следующие результаты:

Направление	Результат	Ссылка
Git-репозиторий	Репозиторий создан, ведётся контроль версий	репозиторий
Статический сайт	Сайт на HTML и CSS, 5 страниц, графическое оформление	сайт
Документация Markdown	Описание проекта, журнал, инструкции	документация
Telegram-бот	Код написан, локально протестирован	телеграмм бот
Отчёт по практике	DOCX, загружены в репозиторий и СДО	отчет

Освоенные навыки: командная разработка (Git, pull request, code review), Markdown, HTML и CSS, Python (FastAPI, SQLAlchemy), TypeScript (React), проектирование REST API, работа с PostgreSQL и S3, Docker, Coolify, CI/CD, Figma, Swagger/OpenAPI.

ЗАКЛЮЧЕНИЕ

В ходе проектной практики выполнено полное задание: реализована базовая часть (Git, Markdown, статический сайт, отчёт) и вариативная часть (разработка Telegram-бота на Python). Код бота написан, загружен в репозиторий, основные функции протестированы локально. Запуск на удалённом сервере требует дополнительной настройки.

Автором отчёта лично обеспечены: Логирование и просмотр ошибок на стенде, шаблон env для новых сервисов, шаблон PR и чеклист ревью, иконки для empty state, а также многочисленные исследования продуктов и интеграций для проекта. Помимо этого был разработан телеграмм-бот с функционалом ai-помощника. Остальные функциональные компоненты (ML-пайплайн, дизайн, аналитика, маркетинг) реализованы другими участниками команды в рамках распределения ролей.

Таким образом, вклад автора является значимым и охватывает ключевые направления архитектуры и инфраструктуры. Для университета ценность работы заключается в готовой платформе, которая уже сейчас позволяет преподавателям автоматизировать создание конспектов из лекций, а студенческим командам — организовать проектную деятельность в едином инструменте.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. FastAPI — документация. FastAPI. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://fastapi.tiangolo.com/>.
2. React — документация. React. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://react.dev/>.
3. Git — документация. Git SCM. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://git-scm.com/doc>.
4. Markdown Guide. Markdown Guide. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://www.markdownguide.org/>.
5. SQLAlchemy — документация. SQLAlchemy. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://docs.sqlalchemy.org/>.
6. Alembic — документация. Alembic. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://alembic.sqlalchemy.org/>.
7. DockerHub — документация. Docker Inc. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://docs.docker.com/>.
8. OpenAI Whisper. GitHub. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://github.com/openai/whisper>.
9. TypeScript — документация. Microsoft. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://www.typescriptlang.org/docs/>.
10. PostgreSQL — документация. The PostgreSQL Global Development Group. [В Интернете] 2026 г. [Цитировано: 20.05.2026 г.] <https://www.postgresql.org/docs/>.

ПРИЛОЖЕНИЕ А

Листинг А.1 — index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
<title>SPARQ — Платформа для студентов Мосполитеха</title>
<link
href="https://fonts.googleapis.com/css2?family=Syne:wght@400;60
0;700;800&family=JetBrains+Mono:wght@400;500&family=Manrope:wgh
t@300;400;500;600&display=swap" rel="stylesheet">
<style>
  :root {
    --bg: #0a0c10;
    --bg2: #0f1117;
    --bg3: #141720;
    --surface: #1a1e28;
    --surface2: #222738;
    --accent: #5b8cff;
    --accent2: #3dffc2;
    --accent3: #ff6b6b;
    --text: #e8ecf4;
    --muted: #7a8299;
    --border: rgba(91,140,255,0.15);
    --glow: 0 0 40px rgba(91,140,255,0.15);
  }

  * { margin: 0; padding: 0; box-sizing: border-box; }
  html { scroll-behavior: smooth; }

  body {
    background: var(--bg);
    color: var(--text);
    font-family: 'Manrope', sans-serif;
    font-size: 15px;
    line-height: 1.7;
    overflow-x: hidden;
  }

  body::before {
    content: '';
    position: fixed;
    inset: 0;
    background-image:
      linear-gradient(rgba(91,140,255,0.04) 1px, transparent
1px),
```

```

        linear-gradient(90deg, rgba(91,140,255,0.04) 1px,
transparent 1px);
        background-size: 60px 60px;
        pointer-events: none;
        z-index: 0;
    }

    .orb {
        position: fixed;
        border-radius: 50%;
        pointer-events: none;
        z-index: 0;
        filter: blur(80px);
    }

    .orb-1 {
        width: 500px; height: 500px;
        background: rgba(91,140,255,0.08);
        top: -200px; right: -100px;
    }

    .orb-2 {
        width: 400px; height: 400px;
        background: rgba(61,255,194,0.05);
        bottom: 20%; left: -100px;
    }

    /* NAV */
    nav {
        position: fixed;
        top: 0; left: 0; right: 0;
        z-index: 100;
        display: flex;
        align-items: center;
        justify-content: space-between;
        padding: 18px 60px;
        background: rgba(10,12,16,0.9);
        backdrop-filter: blur(20px);
        border-bottom: 1px solid var(--border);
    }

    .nav-logo {
        font-family: 'Syne', sans-serif;
        font-weight: 800;
        font-size: 22px;
        letter-spacing: 2px;
        color: var(--accent);
        text-decoration: none;
        cursor: pointer;
    }

    .nav-logo span { color: var(--accent2); }

    nav ul {

```

```

display: flex;
list-style: none;
gap: 32px;
}

nav ul li a {
  color: var(--muted);
  text-decoration: none;
  font-size: 13px;
  font-weight: 500;
  letter-spacing: 0.5px;
  transition: color 0.2s;
  font-family: 'JetBrains Mono', monospace;
  cursor: pointer;
}

nav ul li a:hover,
nav ul li a.active { color: var(--accent); }

.nav-burger {
  display: none;
  flex-direction: column;
  gap: 5px;
  cursor: pointer;
  padding: 4px;
}

.nav-burger span {
  display: block;
  width: 24px;
  height: 2px;
  background: var(--muted);
  transition: all 0.3s;
}

.mobile-menu {
  display: none;
  position: fixed;
  top: 65px; left: 0; right: 0;
  background: rgba(10,12,16,0.97);
  border-bottom: 1px solid var(--border);
  padding: 20px 24px;
  z-index: 99;
  flex-direction: column;
  gap: 16px;
}

.mobile-menu.open { display: flex; }
.mobile-menu a {
  color: var(--muted);
  text-decoration: none;
  font-family: 'JetBrains Mono', monospace;
  font-size: 14px;

```



```

padding: 8px 0;
border-bottom: 1px solid var(--border);
cursor: pointer;
}
.mobile-menu a:hover { color: var(--accent); }

/* PAGES */
.page {
display: none;
position: relative;
z-index: 1;
min-height: 100vh;
padding-top: 80px;
}
.page.active { display: block; }

/* CONTAINERS */
.container {
max-width: 1100px;
margin: 0 auto;
padding: 60px 40px;
}

/* HERO PAGE */
.hero {
min-height: calc(100vh - 80px);
display: flex;
flex-direction: column;
justify-content: center;
text-align: center;
padding: 60px 40px 40px;
max-width: 1100px;
margin: 0 auto;
}

.hero-badge {
display: inline-flex;
align-items: center;
gap: 8px;
background: rgba(61,255,194,0.08);
border: 1px solid rgba(61,255,194,0.25);
color: var(--accent2);
font-family: 'JetBrains Mono', monospace;
font-size: 12px;
padding: 6px 16px;
border-radius: 100px;
margin-bottom: 32px;
animation: fadeUp 0.6s ease both;
}

.hero-badge-dot {

```

```

width: 6px;
height: 6px;
border-radius: 50%;
background: var(--accent2);
animation: pulse 2s infinite;
flex-shrink: 0;
}

@keyframes pulse {
  0%, 100% { opacity: 1; transform: scale(1); }
  50% { opacity: 0.3; transform: scale(0.8); }
}

.hero h1 {
  font-family: 'Syne', sans-serif;
  font-size: clamp(52px, 8vw, 96px);
  font-weight: 800;
  line-height: 0.95;
  letter-spacing: -3px;
  animation: fadeUp 0.6s 0.1s ease both;
  margin-bottom: 8px;
}

.hero h1 .line1 { color: var(--text); display: block; }
.hero h1 .line2 {
  display: block;
  background: linear-gradient(135deg, var(--accent), var(--
accent2));
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

.hero-sub {
  font-size: 18px;
  color: var(--muted);
  max-width: 560px;
  margin: 28px auto 0;
  font-weight: 300;
  animation: fadeUp 0.6s 0.2s ease both;
}

.hero-cta {
  display: flex;
  gap: 16px;
  justify-content: center;
  margin-top: 48px;
  animation: fadeUp 0.6s 0.3s ease both;
  flex-wrap: wrap;
}

```

```

.btn {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 14px 32px;
  border-radius: 8px;
  font-family: 'JetBrains Mono', monospace;
  font-size: 13px;
  font-weight: 500;
  text-decoration: none;
  transition: all 0.2s;
  cursor: pointer;
  border: none;
}

.btn-primary {
  background: var(--accent);
  color: #fff;
  box-shadow: 0 0 32px rgba(91,140,255,0.4);
}

.btn-primary:hover {
  background: #7aa3ff;
  box-shadow: 0 0 48px rgba(91,140,255,0.6);
  transform: translateY(-2px);
}

.btn-outline {
  background: transparent;
  color: var(--muted);
  border: 1px solid var(--border);
}

.btn-outline:hover {
  color: var(--text);
  border-color: rgba(91,140,255,0.4);
  background: rgba(91,140,255,0.05);
}

.btn-page {
  background: transparent;
  color: var(--muted);
  border: 1px solid var(--border);
}

.btn-page:hover {
  color: var(--accent2);
  border-color: rgba(61,255,194,0.3);
  background: rgba(61,255,194,0.05);
}

/* STATS */
.stats-row {
  display: grid;

```

```

    grid-template-columns: repeat(4, 1fr);
    gap: 1px;
    background: var(--border);
    border: 1px solid var(--border);
    border-radius: 12px;
    overflow: hidden;
    margin-top: 80px;
    animation: fadeUp 0.6s 0.4s ease both;
}

.stat-item {
    background: var(--bg2);
    padding: 28px 24px;
    text-align: center;
    transition: background 0.2s;
}

.stat-item:hover { background: var(--surface); }

.stat-num {
    font-family: 'Syne', sans-serif;
    font-size: 36px;
    font-weight: 800;
    color: var(--accent);
    display: block;
    letter-spacing: -1px;
}

.stat-label {
    font-size: 12px;
    color: var(--muted);
    margin-top: 4px;
    font-family: 'JetBrains Mono', monospace;
}

@keyframes fadeUp {
    from { opacity: 0; transform: translateY(24px); }
    to { opacity: 1; transform: translateY(0); }
}

/* SECTION HEADERS */
.section-tag {
    font-family: 'JetBrains Mono', monospace;
    font-size: 12px;
    color: var(--accent2);
    letter-spacing: 2px;
    text-transform: uppercase;
    margin-bottom: 16px;
    display: flex;
    align-items: center;
    gap: 12px;
}

```

```

.section-tag::before {
  content: '';
  width: 32px;
  height: 1px;
  background: var(--accent2);
}

h2 {
  font-family: 'Syne', sans-serif;
  font-size: clamp(32px, 4vw, 52px);
  font-weight: 800;
  letter-spacing: -2px;
  line-height: 1;
  margin-bottom: 20px;
}

.section-desc {
  color: var(--muted);
  font-size: 16px;
  max-width: 560px;
  font-weight: 300;
}

/* ABOUT PAGE */
.content-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 60px;
  margin-top: 60px;
  align-items: start;
}

.about-text p {
  color: var(--muted);
  margin-bottom: 20px;
  font-size: 15px;
  line-height: 1.8;
}

.about-text p strong { color: var(--text); }

.stack-pills {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  margin-top: 32px;
}

.pill {
  font-family: 'JetBrains Mono', monospace;
  font-size: 11px;
  padding: 5px 12px;

```

```

border-radius: 100px;
border: 1px solid var(--border);
color: var(--muted);
background: var(--surface);
transition: all 0.2s;
}
.pill:hover {
  color: var(--accent);
  border-color: rgba(91,140,255,0.4);
}
.pill.accent {
  color: var(--accent);
  border-color: rgba(91,140,255,0.3);
  background: rgba(91,140,255,0.08);
}

/* PIPELINE */
.pipeline {
  display: flex;
  align-items: center;
  gap: 0;
  margin-top: 40px;
  flex-wrap: wrap;
  justify-content: center;
}

.pipe-step {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 12px 18px;
  text-align: center;
  font-family: 'JetBrains Mono', monospace;
  font-size: 11px;
  color: var(--muted);
  min-width: 90px;
  transition: all 0.2s;
}
.pipe-step:hover {
  background: var(--surface2);
  color: var(--accent);
  border-color: rgba(91,140,255,0.3);
}
.pipe-step span {
  display: block;
  font-size: 18px;
  margin-bottom: 4px;
  font-style: normal;
}

.pipe-arrow {

```

```

    color: var(--accent);
    font-size: 18px;
    padding: 0 8px;
    opacity: 0.5;
}

/* MODULES (inside about) */
.modules-inner {
    display: flex;
    flex-direction: column;
    gap: 12px;
    margin-top: 24px;
}

.module-card {
    background: var(--bg2);
    border: 1px solid var(--border);
    border-radius: 12px;
    padding: 20px 24px;
    transition: all 0.3s;
    position: relative;
    overflow: hidden;
}

.module-card::before {
    content: '';
    position: absolute;
    top: 0; left: 0; right: 0;
    height: 2px;
    background: linear-gradient(90deg, var(--accent), var(--
accent2));
    opacity: 0;
    transition: opacity 0.3s;
}

.module-card:hover {
    border-color: rgba(91,140,255,0.3);
    transform: translateY(-2px);
    box-shadow: var(--glow);
}

.module-card:hover::before { opacity: 1; }

.module-card h3 {
    font-family: 'Syne', sans-serif;
    font-size: 16px;
    font-weight: 700;
    margin-bottom: 8px;
    color: var(--text);
}

.module-card p {
    color: var(--muted);
    font-size: 13px;
    line-height: 1.7;
}

```

```

}

/* IMAGE PLACEHOLDER */
.img-placeholder {
  width: 100%;
  border-radius: 12px;
  border: 2px dashed rgba(91,140,255,0.25);
  background: rgba(91,140,255,0.04);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 10px;
  color: var(--muted);
  font-family: 'JetBrains Mono', monospace;
  font-size: 12px;
  text-align: center;
  padding: 40px 20px;
  transition: border-color 0.2s;
  overflow: hidden;
  position: relative;
}

.img-placeholder:hover {
  border-color: rgba(91,140,255,0.45);
}

.img-placeholder .img-icon {
  width: 48px;
  height: 48px;
  border-radius: 10px;
  background: rgba(91,140,255,0.1);
  border: 1px solid rgba(91,140,255,0.2);
  display: flex;
  align-items: center;
  justify-content: center;
}

.img-placeholder .img-icon svg {
  width: 22px;
  height: 22px;
  opacity: 0.5;
  fill: var(--accent);
}

.img-placeholder .img-label {
  color: var(--accent);
  font-size: 11px;
  opacity: 0.7;
}

.img-placeholder .img-hint {
  font-size: 10px;
  opacity: 0.4;
  max-width: 200px;
  line-height: 1.5;
}

```



```

}

/* img-placeholder разные размеры */
.img-placeholder.sm { min-height: 180px; }
.img-placeholder.md { min-height: 260px; }
.img-placeholder.lg { min-height: 360px; }

/* TEAM PAGE */
.lead-card {
  background: linear-gradient(135deg, rgba(91,140,255,0.1),
  rgba(61,255,194,0.05));
  border: 1px solid rgba(91,140,255,0.25);
  border-radius: 16px;
  padding: 32px;
  display: flex;
  gap: 24px;
  align-items: center;
  margin-top: 40px;
  margin-bottom: 32px;
}

.lead-avatar {
  width: 64px;
  height: 64px;
  border-radius: 16px;
  background: linear-gradient(135deg, var(--accent), var(--
accent2));
  display: flex;
  align-items: center;
  justify-content: center;
  font-family: 'Syne', sans-serif;
  font-weight: 800;
  font-size: 24px;
  color: #fff;
  flex-shrink: 0;
}

.lead-info h3 {
  font-family: 'Syne', sans-serif;
  font-size: 20px;
  font-weight: 800;
  margin-bottom: 6px;
}

.lead-info p { color: var(--muted); font-size: 14px; line-
height: 1.7; }

.team-score {
  display: inline-flex;
  align-items: center;
  gap: 4px;
  margin-top: 12px;
}

```

```

    font-family: 'JetBrains Mono', monospace;
    font-size: 11px;
    color: var(--accent2);
    background: rgba(61,255,194,0.08);
    border: 1px solid rgba(61,255,194,0.2);
    padding: 3px 10px;
    border-radius: 100px;
}

.team-section-label {
    font-family: 'JetBrains Mono', monospace;
    font-size: 11px;
    color: var(--muted);
    letter-spacing: 2px;
    text-transform: uppercase;
    padding: 6px 0;
    border-bottom: 1px solid var(--border);
    margin-bottom: 16px;
    margin-top: 32px;
}

.team-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 16px;
}

.team-card {
    background: var(--bg2);
    border: 1px solid var(--border);
    border-radius: 12px;
    padding: 24px 20px;
    transition: all 0.3s;
}

.team-card:hover {
    border-color: rgba(91,140,255,0.3);
    transform: translateY(-3px);
    box-shadow: var(--glow);
}

.team-avatar {
    width: 44px;
    height: 44px;
    border-radius: 10px;
    background: linear-gradient(135deg, var(--accent), var(--accent2));
    display: flex;
    align-items: center;
    justify-content: center;
    font-family: 'Syne', sans-serif;
    font-weight: 800;

```

```

    font-size: 15px;
    color: #fff;
    margin-bottom: 14px;
    flex-shrink: 0;
}

.team-card h3 {
    font-family: 'Syne', sans-serif;
    font-size: 14px;
    font-weight: 700;
    margin-bottom: 4px;
    line-height: 1.3;
}

.team-role {
    font-family: 'JetBrains Mono', monospace;
    font-size: 10px;
    color: var(--accent);
    margin-bottom: 10px;
    display: block;
}

.team-contribution {
    font-size: 12px;
    color: var(--muted);
    line-height: 1.6;
}

/* JOURNAL PAGE */
.journal-list {
    margin-top: 60px;
    display: flex;
    flex-direction: column;
    gap: 24px;
}

.journal-entry {
    background: var(--bg2);
    border: 1px solid var(--border);
    border-radius: 12px;
    padding: 32px;
    display: grid;
    grid-template-columns: 80px 1fr;
    gap: 32px;
    transition: all 0.3s;
}

.journal-entry:hover {
    border-color: rgba(91,140,255,0.3);
    box-shadow: var(--glow);
}

```

```

.journal-date {
  font-family: 'JetBrains Mono', monospace;
  font-size: 11px;
  color: var(--muted);
  text-align: right;
  line-height: 1.6;
  padding-top: 4px;
}
.journal-date strong {
  display: block;
  font-size: 22px;
  color: var(--accent);
  font-weight: 400;
}

.journal-content h3 {
  font-family: 'Syne', sans-serif;
  font-size: 20px;
  font-weight: 700;
  margin-bottom: 12px;
  color: var(--text);
}
.journal-content p {
  color: var(--muted);
  font-size: 14px;
  line-height: 1.8;
  margin-bottom: 16px;
}

.journal-tags {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin-top: 16px;
}

.tag {
  font-family: 'JetBrains Mono', monospace;
  font-size: 10px;
  padding: 4px 10px;
  border-radius: 100px;
  background: rgba(91,140,255,0.08);
  border: 1px solid rgba(91,140,255,0.2);
  color: var(--accent);
}
.tag.green {
  background: rgba(61,255,194,0.08);
  border-color: rgba(61,255,194,0.2);
  color: var(--accent2);
}

```

```

/* journal image block */
.journal-img-row {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
  margin-top: 20px;
}

/* RESOURCES PAGE */
.resources-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
  margin-top: 60px;
}

.resource-card {
  background: var(--bg2);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 24px;
  text-decoration: none;
  color: inherit;
  display: flex;
  gap: 20px;
  align-items: flex-start;
  transition: all 0.3s;
}

.resource-card:hover {
  border-color: rgba(91,140,255,0.35);
  transform: translateY(-3px);
  box-shadow: var(--glow);
}

.resource-icon {
  font-size: 20px;
  flex-shrink: 0;
  width: 44px;
  height: 44px;
  background: var(--surface);
  border-radius: 10px;
  display: flex;
  align-items: center;
  justify-content: center;
  border: 1px solid var(--border);
  font-style: normal;
}

.resource-body h4 {
  font-family: 'Syne', sans-serif;
  font-size: 15px;
}

```

```

    font-weight: 700;
    margin-bottom: 6px;
    color: var(--text);
}
.resource-body p {
    font-size: 13px;
    color: var(--muted);
    line-height: 1.6;
}
.resource-url {
    display: block;
    margin-top: 10px;
    font-family: 'JetBrains Mono', monospace;
    font-size: 11px;
    color: var(--accent);
    opacity: 0.7;
    word-break: break-all;
}

/* SEP */
.sep {
    height: 1px;
    background: var(--border);
    margin: 0 40px;
    position: relative;
    z-index: 1;
}

/* FOOTER */
footer {
    position: relative;
    z-index: 1;
    border-top: 1px solid var(--border);
    padding: 40px 60px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    color: var(--muted);
    font-size: 13px;
    font-family: 'JetBrains Mono', monospace;
}

.footer-logo {
    font-family: 'Syne', sans-serif;
    font-size: 18px;
    font-weight: 800;
    color: var(--accent);
    letter-spacing: 2px;
}

/* REVEAL ANIMATION */

```

```

.reveal {
  opacity: 0;
  transform: translateY(30px);
  transition: opacity 0.6s ease, transform 0.6s ease;
}
.reveal.visible {
  opacity: 1;
  transform: none;
}

/* PAGE TRANSITION */
.page {
  animation: pageIn 0.35s ease both;
}
@keyframes pageIn {
  from { opacity: 0; transform: translateY(16px); }
  to { opacity: 1; transform: translateY(0); }
}

/* RESPONSIVE */
@media (max-width: 900px) {
  nav { padding: 16px 24px; }
  nav ul { display: none; }
  .nav-burger { display: flex; }
  .container { padding: 40px 24px; }
  .hero { padding: 40px 24px; }
  .stats-row { grid-template-columns: repeat(2, 1fr); }
  .content-grid { grid-template-columns: 1fr; gap: 32px; }
  .team-grid { grid-template-columns: repeat(2, 1fr); }
  .resources-grid { grid-template-columns: 1fr; }
  footer { flex-direction: column; gap: 12px; text-align: center; padding: 32px 24px; }
  .lead-card { flex-direction: column; align-items: flex-start; }
  .journal-img-row { grid-template-columns: 1fr; }
}

@media (max-width: 600px) {
  .team-grid { grid-template-columns: 1fr; }
  .journal-entry { grid-template-columns: 1fr; gap: 12px; }
  .journal-date { text-align: left; }
  .hero h1 { letter-spacing: -2px; }
  .stats-row { grid-template-columns: 1fr 1fr; }
}
</style>
</head>
<body>

<div class="orb orb-1"></div>
<div class="orb orb-2"></div>

```

```

<!-- NAV -->
<nav>
  <a class="nav-logo"
onclick="showPage('home')">SP<span>AR</span>Q</a>
  <ul>
    <li><a onclick="showPage('home')" data-
page="home">Главная</a></li>
    <li><a onclick="showPage('about')" data-page="about">0
проекте</a></li>
    <li><a onclick="showPage('team')" data-
page="team">Участники</a></li>
    <li><a onclick="showPage('journal')" data-
page="journal">Журнал</a></li>
    <li><a onclick="showPage('resources')" data-
page="resources">Ресурсы</a></li>
  </ul>
  <div class="nav-burger" onclick="toggleMobile()" id="burger">
    <span></span><span></span><span></span>
  </div>
</nav>

<div class="mobile-menu" id="mobileMenu">
  <a onclick="showPage('home'); closeMobile()">Главная</a>
  <a onclick="showPage('about'); closeMobile()">0 проекте</a>
  <a onclick="showPage('team'); closeMobile()">Участники</a>
  <a onclick="showPage('journal'); closeMobile()">Журнал</a>
  <a onclick="showPage('resources'); closeMobile()">Ресурсы</a>
</div>

<!--
===== --
>
<!-- PAGE: HOME -->
<!--
===== --
>
<div class="page active" id="page-home">
  <div class="hero">
    <div style="display:flex; justify-content:center;">
      <div class="hero-badge">
        <div class="hero-badge-dot"></div>
        Мосполитех · Проектная деятельность · 2025-2026
      </div>
    </div>

    <h1>
      <span class="line1">Платформа</span>
      <span class="line2">SPARQ</span>
    </h1>

    <p class="hero-sub">

```



```

    AI-платформа для расшифровки лекций, управления задачами
    и командной коммуникации внутри Московского Политеха
  </p>

  <div class="hero-cta">
    <a href="http://frontend.94.241.169.118.sslip.io/welcome"
target="_blank" class="btn btn-primary">Открыть платформу</a>
    <a href="http://backend.94.241.169.118.sslip.io/docs"
target="_blank" class="btn btn-outline">Swagger API</a>
    <a href="https://github.com/sparq-platform"
target="_blank" class="btn btn-outline">GitHub</a>
  </div>

  <div class="stats-row">
    <div class="stat-item">
      <span class="stat-num">30+</span>
      <div class="stat-label">участников команды</div>
    </div>
    <div class="stat-item">
      <span class="stat-num">146</span>
      <div class="stat-label">задач выполнено</div>
    </div>
    <div class="stat-item">
      <span class="stat-num">3</span>
      <div class="stat-label">модуля платформы</div>
    </div>
    <div class="stat-item">
      <span class="stat-num">MVP</span>
      <div class="stat-label">запущен в прод</div>
    </div>
  </div>
</div>

<!-- Скриншот главной страницы платформы -->
<div class="container" style="padding-top: 0;">
  
</div>

<footer>
  <div class="footer-logo">SPARQ</div>
  <div>Мосполитех · Проектная деятельность 2025-2026</div>
  <div style="font-size:11px; opacity:0.5">v2.0 · Task
Manager + Messenger</div>
</footer>
</div>
<!--
===== --
>
<!-- PAGE: ABOUT -->

```

```

<!--
===== --
>
<div class="page" id="page-about">
  <div class="container">
    <div class="section-tag">О проекте</div>
    <h2>Что такое<br>SPARQ?</h2>
    <p class="section-desc">От расшифровки лекций до
    полноценной рабочей платформы с Task Manager и Messenger</p>

    <div class="content-grid">
      <div class="about-text reveal">
        <p>
          <strong>SPARQ</strong> – студенческий IT-продукт,
          разработанный в рамках проектной деятельности Московского
          Политехнического университета. Изначально проект стартовал как
          сервис <strong>расшифровки видеолекций</strong> с помощью
          искусственного интеллекта: студенты и преподаватели могли
          автоматически получать текст, конспекты и ключевые тезисы из
          учебных видео.
        </p>
        <p>
          В процессе работы платформа выросла в полноценную
          <strong>рабочую среду</strong>: появился модуль управления
          задачами с kanban-досками и мессенджер с каналами и личными
          сообщениями – всё в фирменном стиле Мосполитеха.
        </p>
        <p>
          Проект полностью разработан командой студентов.
          Фронтенд – <strong>React + TypeScript</strong>, бэкенд –
          <strong>FastAPI + PostgreSQL + S3</strong>, ML-пайплайн
          строится на speech-to-text моделях. Задеплоено через
          <strong>Coolify + Docker</strong> с автоматическим CI/CD.
        </p>
        <div class="stack-pills">
          <span class="pill accent">React</span>
          <span class="pill accent">TypeScript</span>
          <span class="pill accent">FastAPI</span>
          <span class="pill accent">PostgreSQL</span>
          <span class="pill accent">S3</span>
          <span class="pill accent">Docker</span>
          <span class="pill">Whisper STT</span>
          <span class="pill">Sber STT</span>
          <span class="pill">Coolify</span>
          <span class="pill">GitHub Actions</span>
          <span class="pill">Figma</span>
          <span class="pill">JWT Auth</span>
        </div>
      </div>
    </div>

    <div class="reveal" style="transition-delay:0.1s">

```

```

        <div class="pipeline">
            <div class="pipe-
step"><span>Видео</span></div>
            <div class="pipe-arrow"></div>
            <div class="pipe-
step"><span>Аудио</span></div>
            <div class="pipe-arrow"></div>
            <div class="pipe-step"><span>STT
AI</span></div>
            <div class="pipe-arrow"></div>
            <div class="pipe-
step"><span>Текст</span></div>
        </div>

        <div class="modules-inner">
            <div class="module-card">
                <h3>ML — Расшифровка лекций</h3>
                <p>Загрузи видео — получи транскрипт, теги и
структурированный конспект. End-to-end пайплайн с поддержкой
длинных лекций.</p>
            </div>
            <div class="module-card">
                <h3>Task Manager</h3>
                <p>Kanban-доски с drag&drop, карточками задач,
assignee, дедлайнами и тегами. Полная интеграция с REST
API.</p>
            </div>
            <div class="module-card">
                <h3>Messenger</h3>
                <p>Мессенджер с каналами, личными сообщениями,
real-time уведомлениями и поиском для командной работы.</p>
            </div>
        </div>

        <!-- Скриншоты интерфейса -->
        <div style="margin-top: 60px;">
            <div class="section-tag">Интерфейс</div>
            <div style="display: grid; grid-template-columns: 1fr 1fr
1fr; gap: 16px; margin-top: 24px;">
                
                
                
            </div>
        </div>
    </div>

```

```

<footer>
  <div class="footer-logo">SPARQ</div>
  <div>Мосполитех · Проектная деятельность 2025-2026</div>
  <div style="font-size:11px; opacity:0.5">v2.0 · Task
Manager + Messenger</div>
</footer>
</div>

<!--
===== --
>
<!-- PAGE: TEAM -->
<!--
===== --
>
<div class="page" id="page-team">
  <div class="container">
    <div class="section-tag">Участники</div>
    <h2>Команда<br>проекта</h2>
    <p class="section-desc">Более 30 студентов разных
направлений: от разработки до дизайна и маркетинга</p>

    <!-- Тимлид -->
    <div class="lead-card reveal">
      <div class="lead-avatar">ПН</div>
      <div class="lead-info">
        <h3>Никитин Платон Алексеевич</h3>
        <p>
          Тимлид проекта · Группа 231-364 · Бигдата<br>
          Отвечает за архитектуру, декомпозицию задач, деплой
ML-сервиса, CI/CD и координацию всех участников. Задеплоил
платформу через Coolify, реализовал автодеплой бэкенда и ML-
сервиса, ведёт защиту проекта.
        </p>
        <span class="team-score">20 баллов</span>
      </div>
    </div>
    <!-- Backend -->
    <div class="team-section-label">Backend</div>
    <div class="team-grid reveal">
      <div class="team-card">
        <div class="team-avatar">КК</div>
        <h3>Крипак Ксения Романовна</h3>
        <span class="team-role">Backend · FastAPI</span>
        <p class="team-contribution">JWT авторизация,
подключение PostgreSQL, интеграция S3, CRUD API колонок и
карточек, reorder, CI с ruff-линтером, миграции,
рефакторинг.</p>
        <span class="team-score">36 баллов</span>
      </div>
      <div class="team-card">

```

```

    <div class="team-avatar">КД</div>
    <h3>Карпов Денис Александрович</h3>
    <span class="team-role">Backend</span>
    <p class="team-contribution">Оптимистичная блокировка
reorder, timezone-контракт для due_state, унификация ошибок
API, endpoint состава доски, логирование и тесты.</p>
    <span class="team-score">28 баллов</span>
  </div>
  <div class="team-card">
    <div class="team-avatar">МА</div>
    <h3>Мельников Алексей Владимирович</h3>
    <span class="team-role">Backend · Task Manager</span>
    <p class="team-contribution">Схема данных
досок/колонок/карточек, CRUD API досок, получение доски с
вложенными данными, рефакторинг модуля.</p>
    <span class="team-score">19 баллов</span>
  </div>
  <div class="team-card">
    <div class="team-avatar">ЧД</div>
    <h3>Чернушин Дмитрий Сергеевич</h3>
    <span class="team-role">Backend · API</span>
    <p class="team-contribution">Healthcheck ML-сервиса,
логирование с таймингами, закрытие пробелов API для ТОП-10
страниц, проверка живости стендов.</p>
    <span class="team-score">19 баллов</span>
  </div>
</div>

<!-- Frontend -->
<div class="team-section-label">Frontend</div>
<div class="team-grid reveal">
  <div class="team-card">
    <div class="team-avatar">ИА</div>
    <h3>Исаев Арслан Эфлатанович</h3>
    <span class="team-role">Frontend · React</span>
    <p class="team-contribution">UI авторизации, интеграция
загрузки видео с API, экран доски с колонками и карточками,
drag&drop, адаптив, фильтрация, инвентаризация роутов.</p>
    <span class="team-score">31 балл</span>
  </div>
  <div class="team-card">
    <div class="team-avatar">ДИ</div>
    <h3>Дьяченко Иван Максимович</h3>
    <span class="team-role">Frontend · React</span>
    <p class="team-contribution">Toast-уведомления,
синхронизация changed_at, сортировка досок, унификация slug
роутов, карточка задачи с редактированием, DnD колонок.</p>
    <span class="team-score">22 балла</span>
  </div>
  <div class="team-card">
    <div class="team-avatar">ПИ</div>

```

```

        <h3>Петраш Илья Дмитриевич</h3>
        <span class="team-role">Frontend · React</span>
        <p class="team-contribution">Роуты и layout Task
Manager, экран списка досок, управление колонками UI, тайминг
ML-пайплайна, README бэкенда.</p>
        <span class="team-score">8 баллов</span>
    </div>
    <div class="team-card">
        <div class="team-avатар">ГР</div>
        <h3>Галиуллин Руслан Альбертович</h3>
        <span class="team-role">Frontend · DnD</span>
        <p class="team-contribution">Drag&drop карточек внутри
и между колонками с optimistic UI и откатом при ошибке.
Восстановление списка исполнителей для участника доски.</p>
        <span class="team-score">6 баллов</span>
    </div>
</div>

<!-- ML -->
<div class="team-section-label">Machine Learning</div>
<div class="team-grid reveal">
    <div class="team-card">
        <div class="team-avатар">УС</div>
        <h3>Уханов Савва Эдуардович</h3>
        <span class="team-role">ML · Пайплайн</span>
        <p class="team-contribution">End-to-end пайплайн
расшифровки видео (I-III итерации), аудиопрепроцессинг, NLP-
теги, интеграция Sber STT, рефакторинг Docker.</p>
        <span class="team-score">ML Lead</span>
    </div>
    <div class="team-card">
        <div class="team-avатар">ЛО</div>
        <h3>Лухнев Олег Дмитриевич</h3>
        <span class="team-role">ML · Backend</span>
        <p class="team-contribution">Пайплайн расшифровки (I-
III итерации), привязка API статусов к БД, API результата и
авторизации к PostgreSQL, интеграция бэка с ML-сервисом.</p>
        <span class="team-score">25 баллов</span>
    </div>
</div>

<!-- Design -->
<div class="team-section-label">Дизайн</div>
<div class="team-grid reveal">
    <div class="team-card">
        <div class="team-avатар">НС</div>
        <h3>Нерсесян Сюзанна Игоревна</h3>
        <span class="team-role">Дизайн · Figma</span>
        <p class="team-contribution">UI-kit компонентов,
референсы и правила UI, аудит и обновление визуального стиля
Task Manager, интеграция авторизации на фронте.</p>

```

```

        <span class="team-score">12 баллов</span>
    </div>
    <div class="team-card">
        <div class="team-avatar">БА</div>
        <h3>Бабкова Анастасия Александровна</h3>
        <span class="team-role">Дизайн · UX</span>
        <p class="team-contribution">Макеты Task Manager и
        Messenger, адаптивы под брейкпоинты, токены типографики и
        spacing, визуальная система статусов, UX-сценарий.</p>
        <span class="team-score">19 баллов</span>
    </div>
    <div class="team-card">
        <div class="team-avatar">ШС</div>
        <h3>Шамара Софья Евгеньевна</h3>
        <span class="team-role">Дизайн · QA</span>
        <p class="team-contribution">Мини UI-гайд, иконки empty
        state, карта состояний ошибок, e2e проверка Task Manager,
        handoff-пакет для редизайна.</p>
        <span class="team-score">21 балл</span>
    </div>
</div>

<!-- Аналитика и маркетинг -->
<div class="team-section-label">Аналитика и маркетинг</div>
<div class="team-grid reveal">
    <div class="team-card">
        <div class="team-avatar">АС</div>
        <h3>Аулова Софья Александровна</h3>
        <span class="team-role">Документация · Маркетинг</span>
        <p class="team-contribution">Глоссарий, хронология
        проекта, one-pager, три версии о проекте, презентация защиты,
        таблица медиаканалов, UI-kit Task Manager, release notes.</p>
        <span class="team-score">22 балла</span>
    </div>
    <div class="team-card">
        <div class="team-avatar">СВ</div>
        <h3>Ситуха Виктор Иванович</h3>
        <span class="team-role">Маркетинг · PM</span>
        <p class="team-contribution">Хук-фразы под роли, анализ
        конкурентов, JTBD-карты для ТМ и Messenger, карты микротекстов,
        сценарии демо, матрицы пользовательских сценариев.</p>
        <span class="team-score">30 баллов</span>
    </div>
    <div class="team-card">
        <div class="team-avatar">СД</div>
        <h3>Сысоев Давид Алексеевич</h3>
        <span class="team-role">Аналитика · Маркетинг</span>
        <p class="team-contribution">Анализ платформ для
        интеграции, оценка сложности, расчёт стоимости разработки и
        поддержки продукта.</p>
        <span class="team-score">40 баллов</span>
    </div>

```

```

    </div>
    <div class="team-card">
      <div class="team-avatar">XM</div>
      <h3>Хамидуллин Михаил Константинович</h3>
      <span class="team-role">Аналитика · Data</span>
      <p class="team-contribution">ERD-диаграмма, продуктовые метрики, Q&A продукта, мини-гайды по использованию, FAQ, интеграция календаря, уведомления.</p>
      <span class="team-score">31 балл</span>
    </div>
    <div class="team-card">
      <div class="team-avatar">XA</div>
      <h3>Хлопцев Александр Алексеевич</h3>
      <span class="team-role">Аналитика · Research</span>
      <p class="team-contribution">Гипотезы улучшения продукта, user research и интервью ЦА, кейсы использования, схема данных тегов и API CRUD тегов.</p>
      <span class="team-score">27 баллов</span>
    </div>
    <div class="team-card">
      <div class="team-avatar">MA</div>
      <h3>Мочалов Александр Александрович</h3>
      <span class="team-role">Документация · Аналитика</span>
      <p class="team-contribution">Словарь полей videos, стек технологий, паспорт проекта, ТОП-10 страниц без бэка, перф-чек нагрузки на чат, документация API.</p>
      <span class="team-score">21 балл</span>
    </div>
    <div class="team-card">
      <div class="team-avatar">КЮ</div>
      <h3>Котельников Юрий Вениаминович</h3>
      <span class="team-role">DevOps · Инфра</span>
      <p class="team-contribution">Логирование и просмотр ошибок на стенде, шаблон env для новых сервисов, шаблон PR и чеклист ревью, иконки для empty state.</p>
      <span class="team-score">9 баллов</span>
    </div>
  </div>
</div>

<footer>
  <div class="footer-logo">SPARQ</div>
  <div>Мосполитех · Проектная деятельность 2025-2026</div>
  <div style="font-size:11px; opacity:0.5">v2.0 · Task Manager + Messenger</div>
</footer>
</div>

<!--
===== --
>

```



```

<!-- PAGE: JOURNAL -->
<!--
===== --
>
<div class="page" id="page-journal">
  <div class="container">
    <div class="section-tag">Журнал прогресса</div>
    <h2>Как развивался<br>проект</h2>
    <p class="section-desc">Ключевые вехи и итерации разработки
SPARQ с октября 2025 по май 2026</p>

    <div class="journal-list">

      <div class="journal-entry reveal">
        <div class="journal-date">
          <strong>ОCT</strong>
          2025
        </div>
        <div class="journal-content">
          <h3>Старт: MVP расшифровки видео</h3>
          <p>
            Команда собрана — больше 30 студентов из разных
            групп. Запущен первый ML-пайплайн для расшифровки видеолекций:
            API принимает видео, извлекает аудио, прогоняет через Whisper
            STT и отдаёт текст. Параллельно разработан UI авторизации на
            React, спроектирована БД в PostgreSQL, созданы базовые REST-
            эндпоинты. Развёрнута база данных на сервере, начато оформление
            Figma-дизайна всех экранов.
          </p>
          <div class="journal-img-row">
            
            
          </div>
          <div class="journal-tags">
            <span class="tag">ML Pipeline v1</span>
            <span class="tag">FastAPI</span>
            <span class="tag">React UI</span>
            <span class="tag green">Завершено</span>
          </div>
        </div>
      </div>

      <div class="journal-entry reveal">
        <div class="journal-date">
          <strong>DEC</strong>
          2025
        </div>
        <div class="journal-content">
          <h3>Интеграция: S3, PostgreSQL, CI/CD</h3>

```

```

    <p>
        Бэкенд полностью переведён на реальную БД и S3-
хранилище. Подключена авторизация через JWT с хранением сессий
в PostgreSQL. ML-сервис переведён на асинхронные задачи –
пользователь видит статус обработки в реальном времени.
Фронтенд интегрирован с API: загрузка видео с прогресс-баром,
страница результатов, экспорт. Настроен Coolify для автодеплойа.
Проведена защита первого семестра.
    </p>
    <div class="journal-img-row">
        
        
    </div>
    <div class="journal-tags">
        <span class="tag">PostgreSQL</span>
        <span class="tag">S3 Storage</span>
        <span class="tag">Docker CI/CD</span>
        <span class="tag">Coolify</span>
        <span class="tag green">Защита прошла</span>
    </div>
</div>
</div>

<div class="journal-entry reveal">
    <div class="journal-date">
        <strong>FEB</strong>
        APR 2026
    </div>
    <div class="journal-content">
        <h3>Task Manager: полный Kanban</h3>
        <p>
            Реализован модуль Task Manager – полноценный аналог
WEEEEK в стиле Мосполитеха. На бэкенде: схема данных досок,
колонок и карточек, CRUD API, reorder с пересчётом позиций,
теги, участники. На фронтенде: kanban-доска с drag&drop внутри
и между колонками, карточки с assignee и дедлайном, быстрое
создание задач, фильтрация. Собран полный UI-kit, проведено
e2e-тестирование.
        </p>
        <div class="journal-img-row">
            
            
        </div>
        <div class="journal-tags">
            <span class="tag">Kanban Board</span>
            <span class="tag">Drag & Drop</span>
            <span class="tag">REST API</span>
        </div>
    </div>
</div>

```

```

        <span class="tag">Figma UI-kit</span>
        <span class="tag green">В продакшене</span>
    </div>
</div>
</div>

<div class="journal-entry reveal">
    <div class="journal-date">
        <strong>APR</strong>
        MAY 2026
    </div>
    <div class="journal-content">
        <h3>Messenger и финальная платформа</h3>
        <p>
            Запущен Messenger с каналами, личными сообщениями и
            инфраструктурой для real-time. Дизайн полностью
            перепроектирован: новые макеты, редизайн Task Manager.
            Проведена инвентаризация API, закрыты все пробелы для ТОП-10
            страниц. Настроены инструкции по логам, шаблоны env. Проект
            подготовлен к итоговой защите.
        </p>
        <div class="journal-img-row">
            
            
        </div>
        <div class="journal-tags">
            <span class="tag">Messenger</span>
            <span class="tag">WebSocket готовность</span>
            <span class="tag">API Coverage</span>
            <span class="tag green">Текущий статус</span>
        </div>
    </div>
</div>

</div>
</div>

<footer>
    <div class="footer-logo">SPARQ</div>
    <div>Мосполитех · Проектная деятельность 2025-2026</div>
    <div style="font-size:11px; opacity:0.5">v2.0 · Task
    Manager + Messenger</div>
</footer>
</div>

<!--
===== --
>
<!-- PAGE: RESOURCES -->

```

```

<!--
===== --
>
<div class="page" id="page-resources">
  <div class="container">
    <div class="section-tag">Ресурсы</div>
    <h2>Полезные<br>ссылки</h2>
    <p class="section-desc">Репозитории, документация, дизайн и
рабочие материалы проекта</p>

    <div class="resources-grid">
      <a href="http://frontend.94.241.169.118.sslip.io/welcome"
target="_blank" class="resource-card">
        <div class="resource-icon">&#127760;</div>
        <div class="resource-body">
          <h4>Платформа SPARQ – Frontend</h4>
          <p>Рабочий стенд: авторизация, загрузка видео, Task
Manager, расшифровки</p>
          <span class="resource-
url">frontend.94.241.169.118.sslip.io</span>
        </div>
      </a>

      <a href="http://backend.94.241.169.118.sslip.io/docs"
target="_blank" class="resource-card">
        <div class="resource-icon">&#9881;</div>
        <div class="resource-body">
          <h4>Swagger – документация API</h4>
          <p>Интерактивная документация всех REST-эндпоинтов:
авторизация, видео, Task Manager, статусы</p>
          <span class="resource-
url">backend.94.241.169.118.sslip.io/docs</span>
        </div>
      </a>

      <a href="https://github.com/sparq-platform/sparq-
frontend" target="_blank" class="resource-card">
        <div class="resource-icon">&#60;/&#62;</div>
        <div class="resource-body">
          <h4>GitHub – Frontend</h4>
          <p>Исходный код фронтенда: React + TypeScript,
компоненты, роуты, интеграция с API</p>
          <span class="resource-url">github.com/sparq-
platform/sparq-frontend</span>
        </div>
      </a>

      <a href="https://github.com/sparq-platform/sparq-backend"
target="_blank" class="resource-card">
        <div class="resource-icon">&#128296;</div>
        <div class="resource-body">

```

```

        <h4>GitHub – Backend</h4>
        <p>FastAPI, SQLAlchemy, миграции Alembic, CI с ruff-
линттером, интеграция S3 и PostgreSQL</p>
        <span class="resource-url">github.com/sparq-
platform/sparq-backend</span>
    </div>
</a>

    <a
href="https://www.figma.com/design/uEPnyxFzl8IioYbYBsna3a/"
target="_blank" class="resource-card">
        <div class="resource-icon">&#9998;</div>
        <div class="resource-body">
            <h4>Figma – Дизайн платформы</h4>
            <p>Полный UI-kit, макеты Task Manager и Messenger,
UI-гайд, компоненты, адаптивы</p>
            <span class="resource-url">figma.com – Платформа
SPARQ</span>
        </div>
    </a>

    <a
href="https://drive.google.com/drive/folders/1xlfR_6djH8NHqD_sq
KwnyIcr4g19n6MP" target="_blank" class="resource-card">
        <div class="resource-icon">&#128193;</div>
        <div class="resource-body">
            <h4>Google Drive – Материалы</h4>
            <p>Все отчёты, документы, аналитика, презентации,
таймлайн, паспорт проекта</p>
            <span class="resource-url">drive.google.com – Проект
SPARQ</span>
        </div>
    </a>

    <a
href="https://docs.google.com/spreadsheets/d/1tbF6xVipkCUIRLome
mlo_v8x7xMA40qdnCx6BiB2GzA" target="_blank" class="resource-
card">
        <div class="resource-icon">&#128202;</div>
        <div class="resource-body">
            <h4>Таблица задач проекта</h4>
            <p>Полный бэклог: 146+ задач по веткам Frontend,
Backend, ML, Design, Marketing с баллами и исполнителями</p>
            <span class="resource-url">docs.google.com – Таблица
SPARQ</span>
        </div>
    </a>

    <a href="https://t.me/+LKQr0EgBan85Y2Yy" target="_blank"
class="resource-card">
        <div class="resource-icon">&#9993;</div>

```

```

        <div class="resource-body">
            <h4>Telegram — Группа проекта</h4>
            <p>Оперативное общение команды, анонсы, обновления
платформы и координация спринтов</p>
            <span class="resource-url">t.me — Группа SPARQ</span>
        </div>
    </a>
</div>
</div>

<footer>
    <div class="footer-logo">SPARQ</div>
    <div>Мосполитех · Проектная деятельность 2025-2026</div>
    <div style="font-size:11px; opacity:0.5">v2.0 · Task
Manager + Messenger</div>
</footer>
</div>

<script>
    // ---- НАВИГАЦИЯ МЕЖДУ СТРАНИЦАМИ ----
    function showPage(name) {
        // скрываем все страницы
        document.querySelectorAll('.page').forEach(p => {
            p.classList.remove('active');
        });

        // показываем нужную
        const target = document.getElementById('page-' + name);
        if (target) {
            target.classList.add('active');
            // убираем предыдущую анимацию и запускаем снова
            target.style.animation = 'none';
            target.offsetHeight; // reflow
            target.style.animation = '';
        }

        // обновляем активную ссылку в навигации
        document.querySelectorAll('nav ul li a').forEach(a => {
            a.classList.toggle('active', a.dataset.page === name);
        });

        // прокручиваем вверх
        window.scrollTo({ top: 0, behavior: 'smooth' });

        // запускаем reveal для новой страницы
        setTimeout(initReveal, 100);
    }

    // ---- МОБИЛЬНОЕ МЕНЮ ----
    function toggleMobile() {

```

```

document.getElementById('mobileMenu').classList.toggle('open');
}
function closeMobile() {
document.getElementById('mobileMenu').classList.remove('open');
}

// ---- SCROLL REVEAL ----
function initReveal() {
  const observer = new IntersectionObserver((entries) => {
    entries.forEach(e => {
      if (e.isIntersecting) {
        e.target.classList.add('visible');
      }
    });
  }, { threshold: 0.08 });

document.querySelectorAll('.reveal:not(.visible)').forEach(el
=> {
  observer.observe(el);
});
}

// ---- STAGGER ДЛЯ КАРТОЧЕК КОМАНДЫ ----
function initTeamStagger() {
  document.querySelectorAll('.team-grid').forEach(grid => {
    grid.querySelectorAll('.team-card').forEach((card, i) =>
{
      card.style.transitionDelay = `${i * 0.05}s`;
      if (!card.classList.contains('reveal')) {
        card.classList.add('reveal');
      }
    });
  });
}

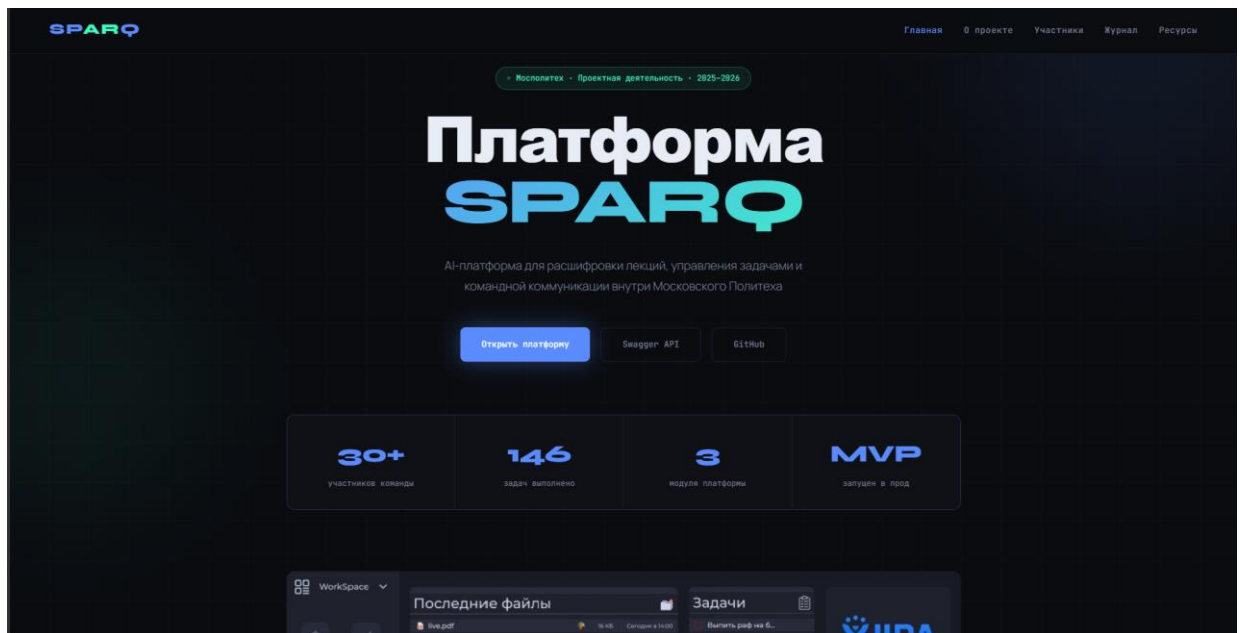
// ---- ИНИЦИАЛИЗАЦИЯ ----
document.addEventListener('DOMContentLoaded', () => {
  initReveal();
  initTeamStagger();

  // закрытие мобильного меню при клике вне
document.addEventListener('click', (e) => {
  const menu = document.getElementById('mobileMenu');
  const burger = document.getElementById('burger');
  if (menu.classList.contains('open') &&
      !menu.contains(e.target) &&
      !burger.contains(e.target)) {
    closeMobile();
  }
});

```

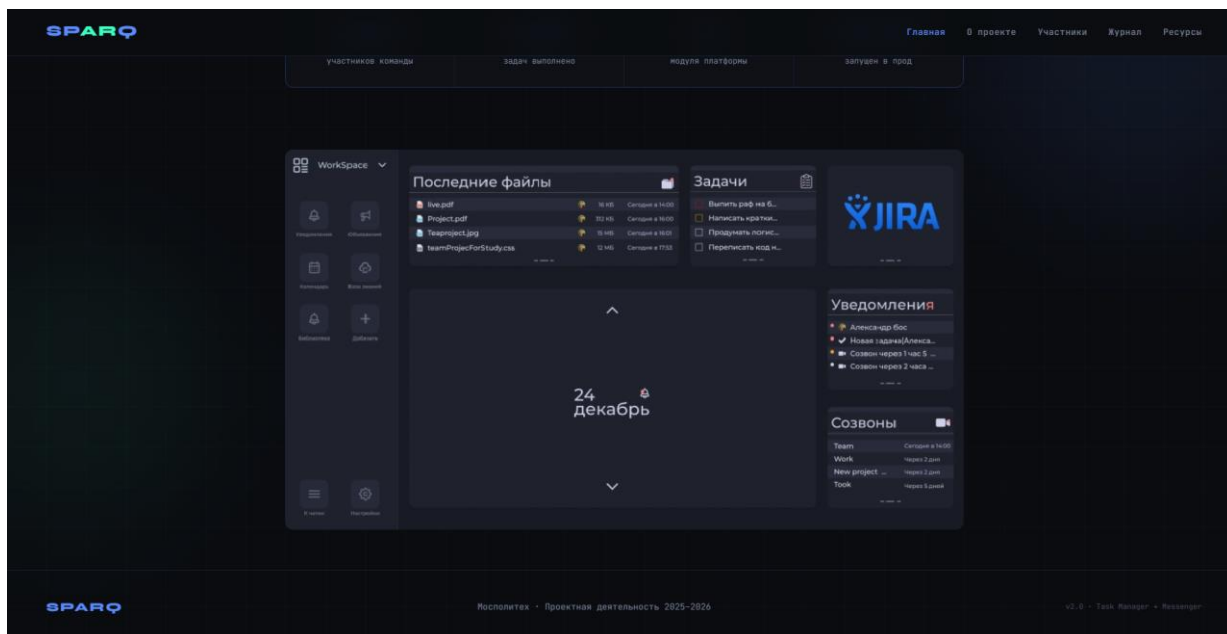
```
    }  
  });  
});  
</script>  
  
</body>  
</html>
```

Скриншот Б.1 — Главная страница сайта проекта SPARQ



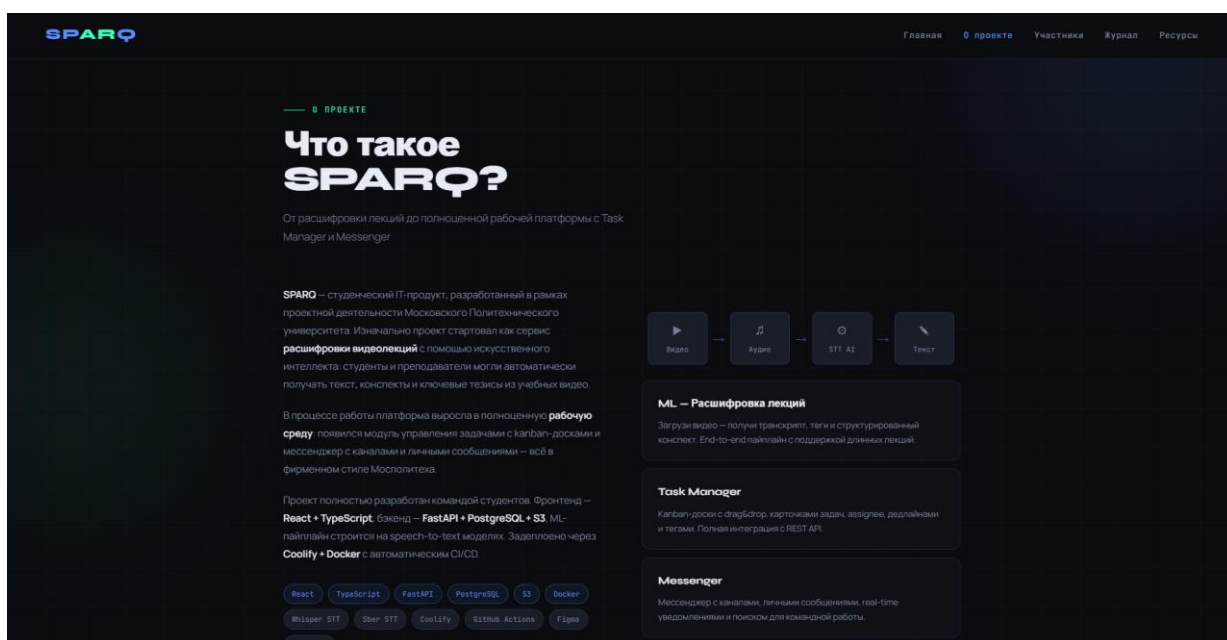
Скриншот Б.1 — «Главная»

Скриншот Б.2 — Главная страница сайта проекта SPARQ



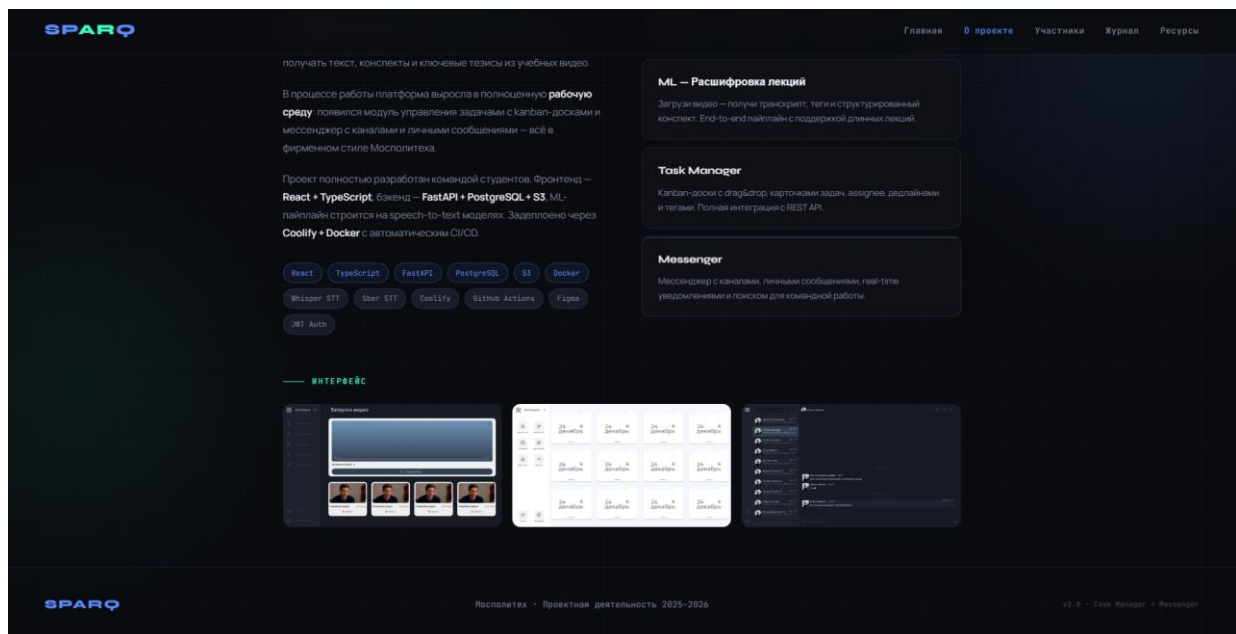
Скриншот Б.2 — «Главная»

Скриншот В.1 — Страница «О проекте»



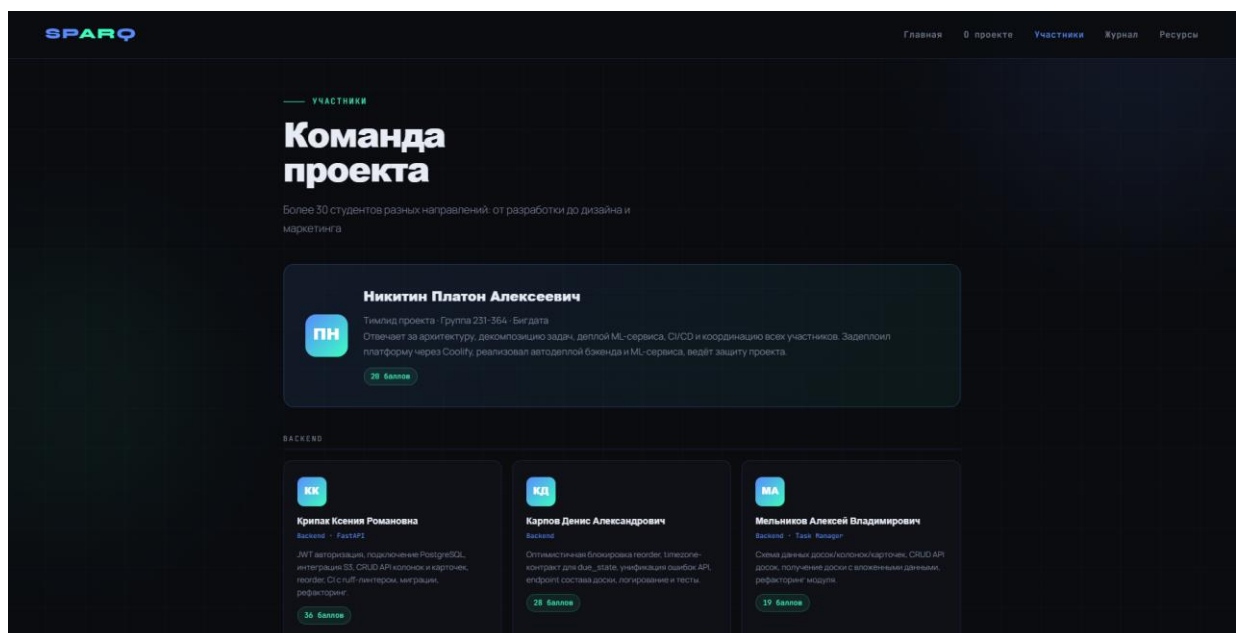
Скриншот В.1 — «О проекте»

Скриншот В.2 — Страница «О проекте»



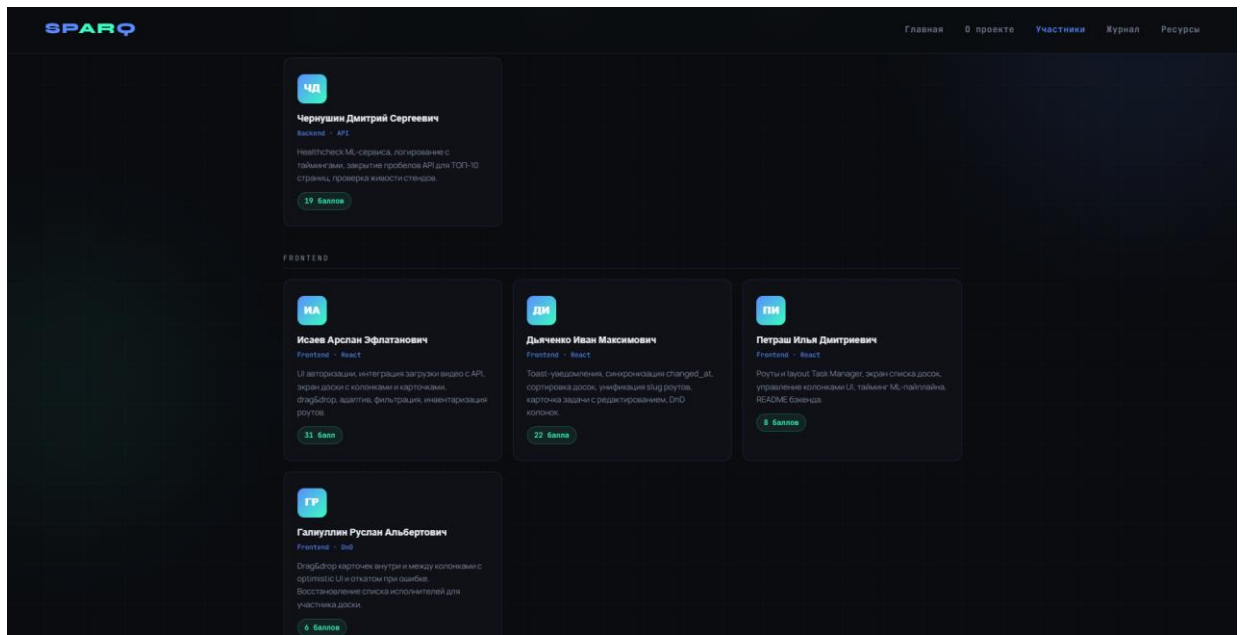
Скриншот В.2 — «О проекте»

Скриншот Г.1 — Страница «Участники», описание вклада каждого участника, выполненные задачи.



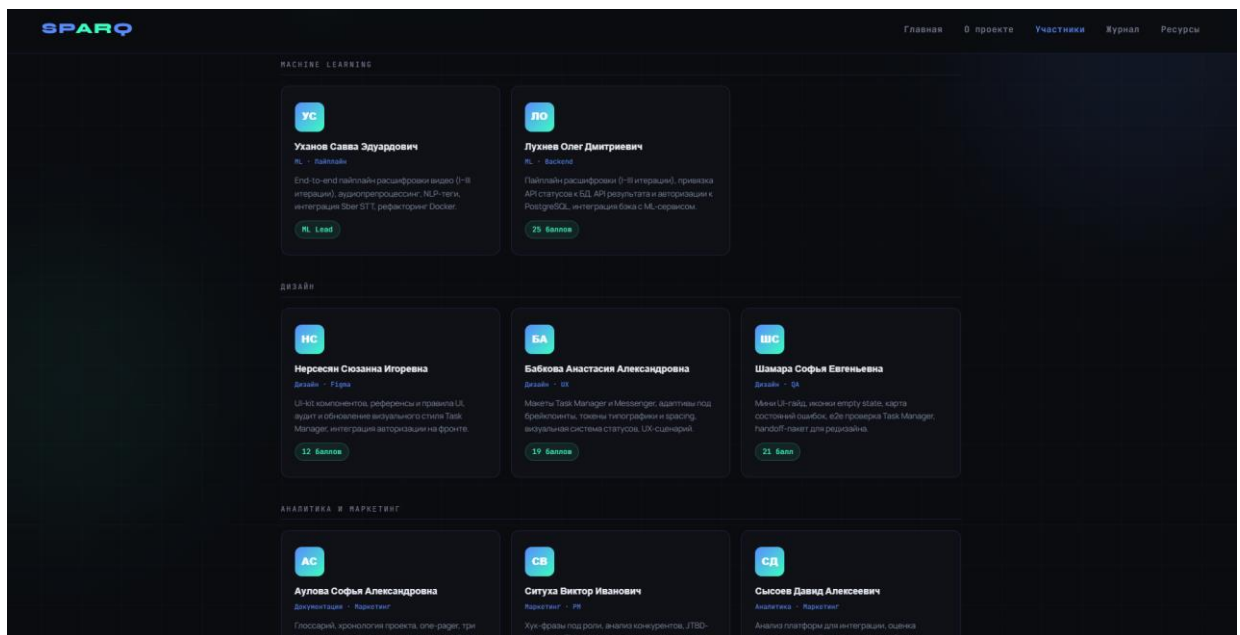
Скриншот Г.1 — «Участники»

Скриншот Г.2 — Страница «Участники», описание вклада каждого участника, выполненные задачи.



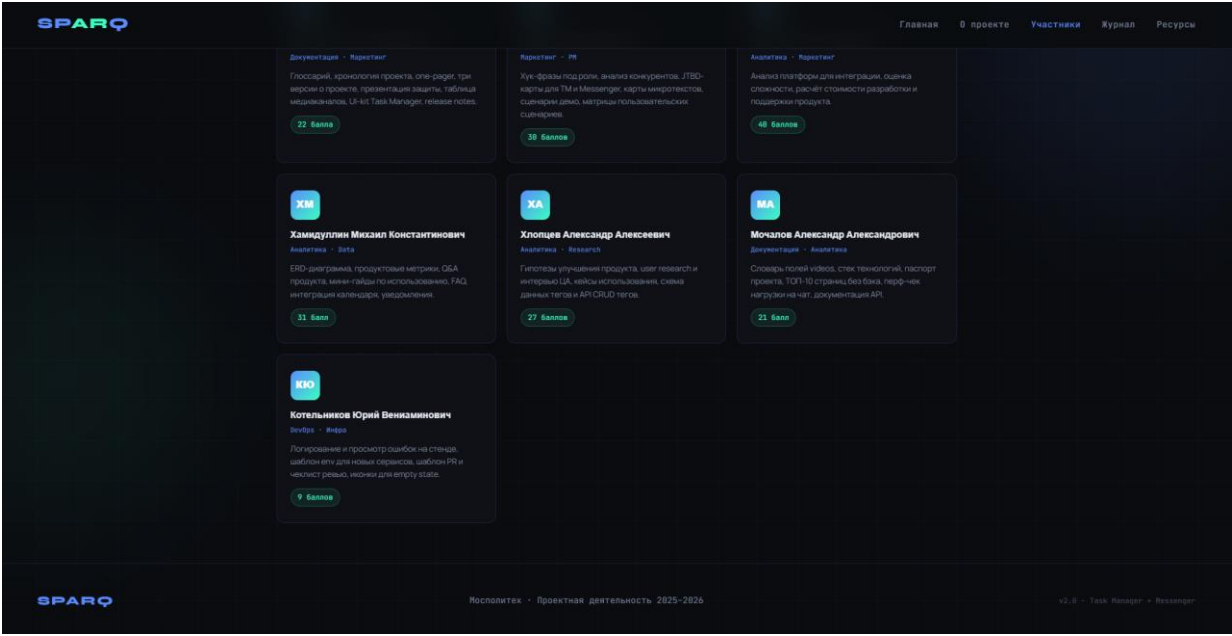
Скриншот Г.2 — «Участники»

Скриншот Г.3 — Страница «Участники», описание вклада каждого участника, выполненные задачи.



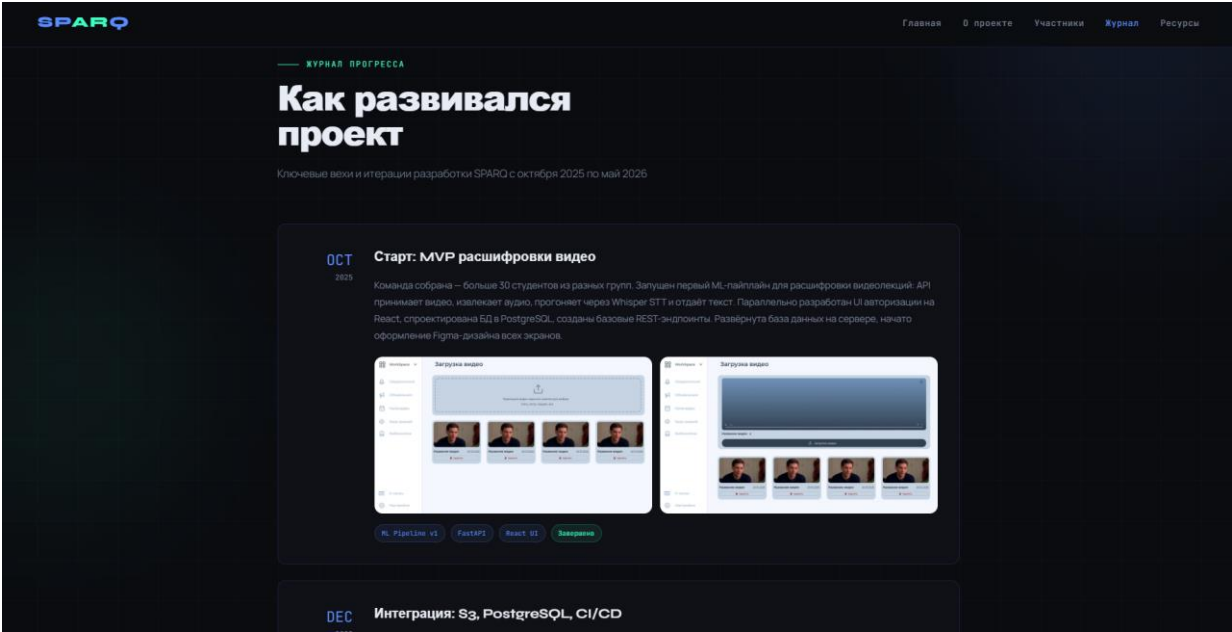
Скриншот Г.3 — «Участники»

Скриншот Г.4 — Страница «Участники», описание вклада каждого участника, выполненные задачи.



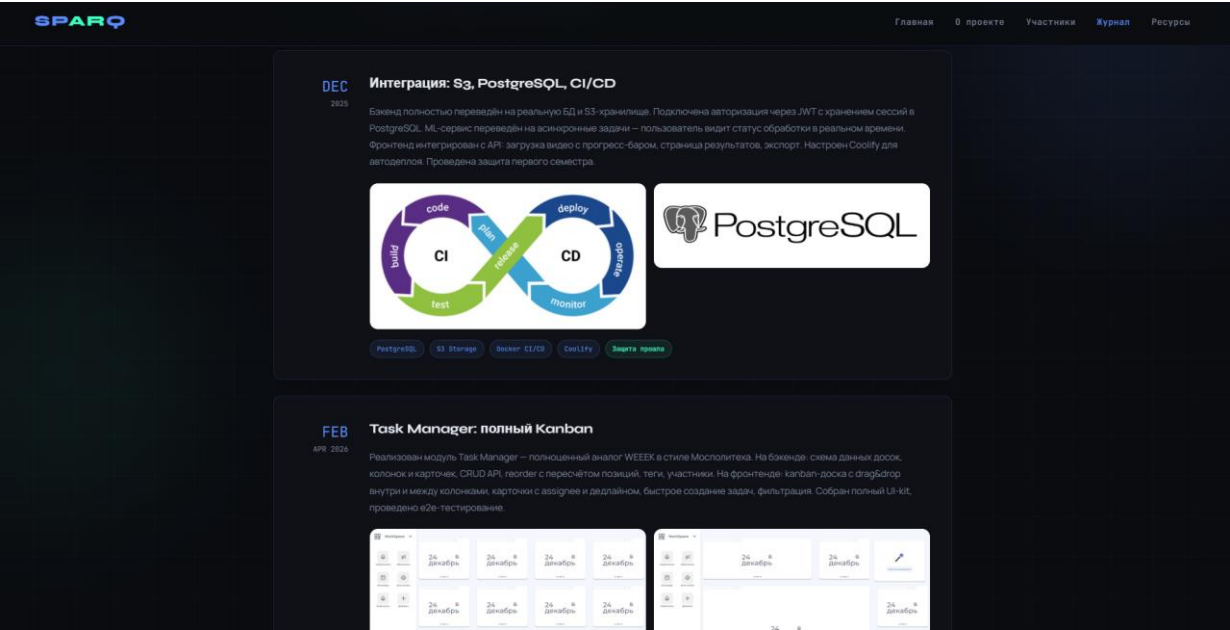
Скриншот Г.4 — «Участники»

Скриншот Д.1 — Страница «Журнал», описание этапов развития проекта.



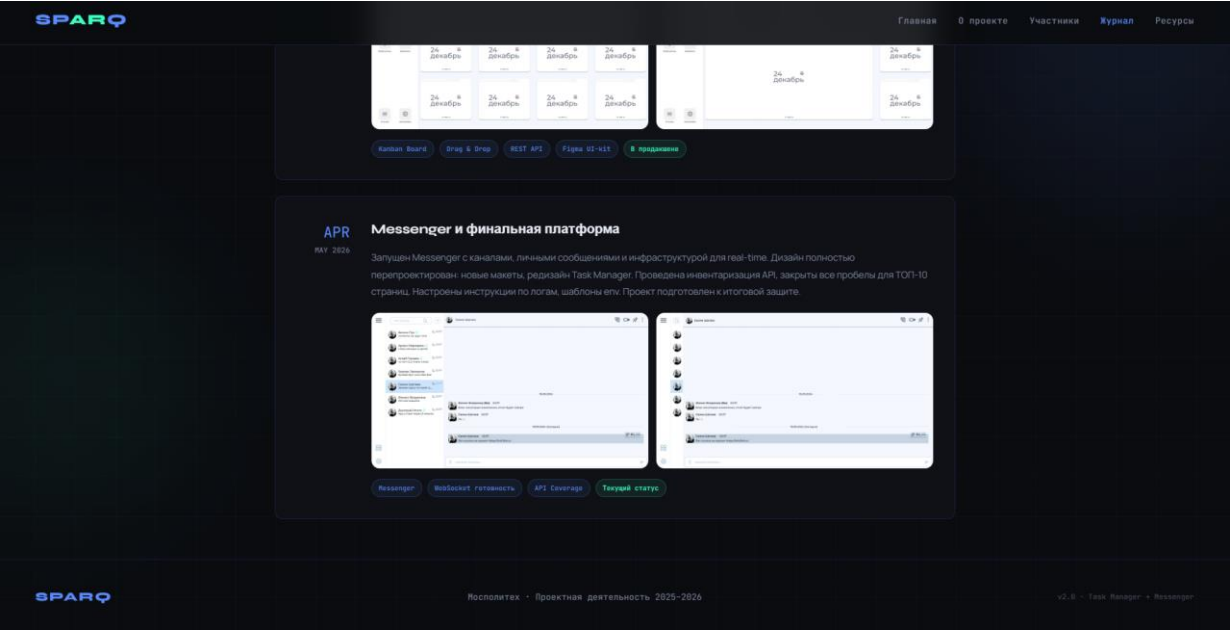
Скриншот Д.1 — «Журнал»

Скриншот Д.2 — Страница «Журнал», описание этапов развития проекта.



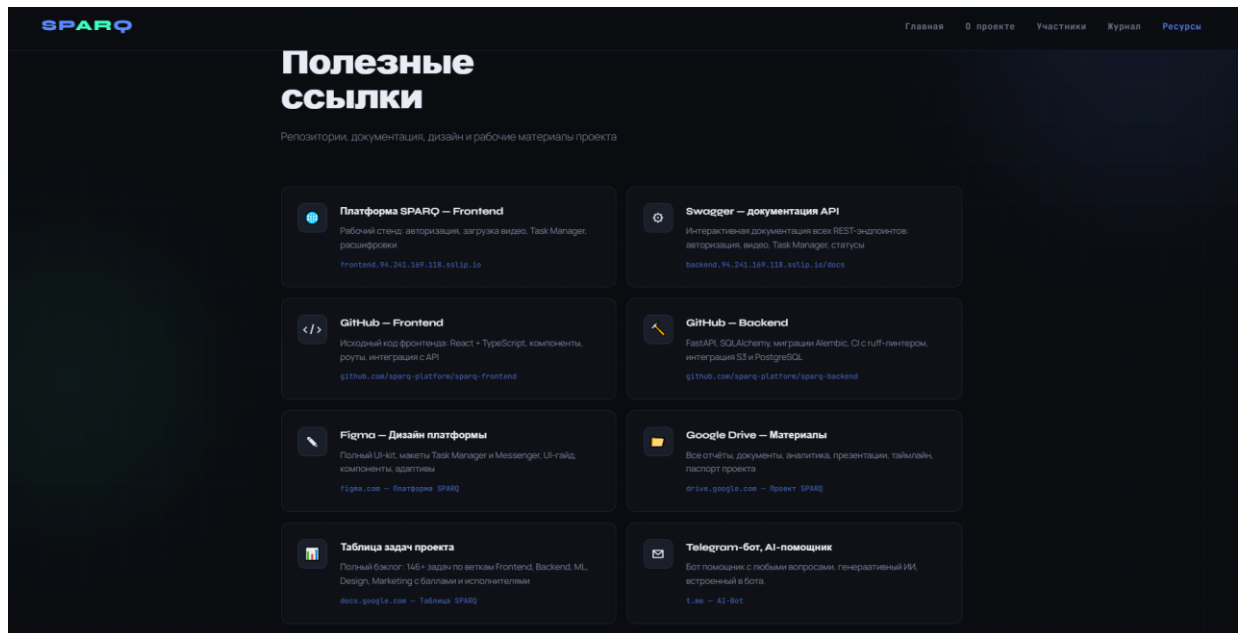
Скриншот Д.2 — «Журнал»

Скриншот Д.3 — Страница «Журнал», описание этапов развития проекта.



Скриншот Д.3 — «Журнал»

Скриншот Е.1 — Страница «Ресурсы», ссылки на платформу, дизайн, тг-бота.



Скриншот Е.1 — «Ресурсы»

ПРИЛОЖЕНИЕ Б

Листинг Ж.1 — bot.py

```
import asyncio
import logging
from aiogram import Bot, Dispatcher, types, F
from aiogram.filters import Command, CommandStart
from aiogram.types import Message, BotCommand
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.fsm.storage.memory import MemoryStorage
from openai import AsyncOpenAI

from config import (
    TELEGRAM_TOKEN,
    OPENAI_API_KEY,
    GPT_MODEL,
    MAX_TOKENS,
    TEMPERATURE,
    MAX_HISTORY_LENGTH,
```

```

)

# Логирование
logging.basicConfig(
    level=logging.INFO,
    format="%%(asctime)s - %(name)s - %(levelname)s - %(message)s",
)
logger = logging.getLogger(__name__)

# Инициализация бота и диспетчера
bot = Bot(token=TELEGRAM_TOKEN)
dp = Dispatcher(storage=MemoryStorage())

# Инициализация OpenAI клиента
client = AsyncOpenAI(api_key=OPENAI_API_KEY)

# Хранилище истории диалогов
user_histories: dict[int, list[dict]] = {}

# =====
# Состояния FSM
# =====
class UserState(StatesGroup):
    chatting = State()
    waiting_system_prompt = State()

# =====
# Вспомогательные функции
# =====
def get_user_history(user_id: int) -> list[dict]:
    """Получить историю диалога пользователя."""
    if user_id not in user_histories:
        user_histories[user_id] = []
    return user_histories[user_id]

def add_to_history(user_id: int, role: str, content: str):
    """Добавить сообщение в историю."""
    history = get_user_history(user_id)
    history.append({"role": role, "content": content})

```

```

# Ограничиваем длину истории
if len(history) > MAX_HISTORY_LENGTH * 2:
    user_histories[user_id] = history[-MAX_HISTORY_LENGTH * 2:]

def clear_history(user_id: int):
    """Очистить историю диалога."""
    user_histories[user_id] = []

async def ask_gpt(
    user_id: int,
    user_message: str,
    system_prompt: str = "Ты полезный ассистент. Отвечай на русском языке.",
) -> str:
    """Отправить запрос к GPT и получить ответ."""
    try:
        add_to_history(user_id, "user", user_message)

        messages = [{"role": "system", "content": system_prompt}]
        messages.extend(get_user_history(user_id))

        response = await client.chat.completions.create(
            model=GPT_MODEL,
            messages=messages,
            max_tokens=MAX_TOKENS,
            temperature=TEMPERATURE,
        )

        assistant_message = response.choices[0].message.content
        add_to_history(user_id, "assistant", assistant_message)

        return assistant_message

    except Exception as e:
        logger.error(f"Ошибка при запросе к GPT: {e}")
        # Удаляем последнее сообщение пользователя из истории при ошибке
        history = get_user_history(user_id)
        if history and history[-1]["role"] == "user":
            history.pop()
        raise

```



```

# =====
# Клавиатуры
# =====
def get_main_keyboard():
    """Главная клавиатура."""
    keyboard = types.ReplyKeyboardMarkup(
        keyboard=[
            [
                types.KeyboardButton(text="🆕 Новый диалог"),
                types.KeyboardButton(text="⚙️ Системный промпт"),
            ],
            [
                types.KeyboardButton(text="📊 Статистика"),
                types.KeyboardButton(text="❓ Помощь"),
            ],
        ],
        resize_keyboard=True,
    )
    return keyboard

def get_cancel_keyboard():
    """Клавиатура отмены."""
    keyboard = types.ReplyKeyboardMarkup(
        keyboard=[[types.KeyboardButton(text="✖ Отмена")]],
        resize_keyboard=True,
    )
    return keyboard

# =====
# Хранилище системных промптов
# =====
user_system_prompts: dict[int, str] = {}
DEFAULT_SYSTEM_PROMPT = "Ты полезный ассистент. Отвечай на русском языке."

def get_system_prompt(user_id: int) -> str:
    """Получить системный промпт пользователя."""
    return user_system_prompts.get(user_id, DEFAULT_SYSTEM_PROMPT)

```

```

# =====
# Обработчики команд
# =====
@dp.message(CommandStart())
async def cmd_start(message: Message, state: FSMContext):
    """Обработчик команды /start."""
    await state.set_state(UserState.chatting)
    user_name = message.from_user.first_name

    await message.answer(
        f"Привет, {user_name}!\n\n"
        "Я — бот с интеграцией GPT. Просто напиши мне что-нибудь, "
        "и я постараюсь помочь!\n\n"
        "📌 Доступные команды:\n"
        "/new — начать новый диалог\n"
        "/help — помощь\n"
        "/stats — статистика\n"
        "/system — изменить системный промпт",
        reply_markup=get_main_keyboard(),
    )

@dp.message(Command("help"))
@dp.message(F.text == "🔍 Помощь")
async def cmd_help(message: Message):
    """Обработчик команды /help."""
    await message.answer(
        "🤖 <b>Как пользоваться ботом:</b>\n\n"
        "1. Просто напишите сообщение — бот ответит с помощью GPT\n"
        "2. Бот помнит контекст диалога\n"
        "3. Используйте <b>Новый диалог</b> для сброса контекста\n\n"
        "📋 <b>Команды:</b>\n"
        "/start — перезапустить бота\n"
        "/new — начать новый диалог\n"
        "/system — изменить системный промпт\n"
        "/stats — показать статистику\n"
        "/help — эта справка\n\n"
        "⚙️ <b>Системный промпт</b> — инструкция для ИИ о его роли и поведении",
        parse_mode="HTML",
        reply_markup=get_main_keyboard(),
    )

```

```

)

@dp.message(Command("new"))
@dp.message(F.text == "🆕 Новый диалог")
async def cmd_new_dialog(message: Message, state: FSMContext):
    """Начать новый диалог."""
    await state.set_state(UserState.chatting)
    clear_history(message.from_user.id)
    await message.answer(
        "🆕 Новый диалог начат! История очищена.\n"
        "Можете задать новый вопрос.",
        reply_markup=get_main_keyboard(),
    )

@dp.message(Command("stats"))
@dp.message(F.text == "📊 Статистика")
async def cmd_stats(message: Message):
    """Показать статистику."""
    user_id = message.from_user.id
    history = get_user_history(user_id)
    messages_count = len(history)
    user_messages = sum(1 for m in history if m["role"] == "user")
    system_prompt = get_system_prompt(user_id)

    await message.answer(
        f"📊 <b>Статистика диалога:</b>\n\n"
        f"Всего сообщений: {messages_count}\n"
        f"Ваших сообщений: {user_messages}\n"
        f"Ответов ИИ: {messages_count - user_messages}\n\n"
        f"⚙️ Текущий системный промпт:\n"
        f"<i>{system_prompt}</i>",
        parse_mode="HTML",
        reply_markup=get_main_keyboard(),
    )

@dp.message(Command("system"))
@dp.message(F.text == "⚙️ Системный промпт")
async def cmd_system(message: Message, state: FSMContext):
    """Изменить системный промпт."""

```

```

current_prompt = get_system_prompt(message.from_user.id)
await state.set_state(UserState.waiting_system_prompt)
await message.answer(
    f"⚙️ <b>Изменение системного промпта</b>\n\n"
    f"Текущий промпт:\n<i>{current_prompt}</i>\n\n"
    "Введите новый системный промпт или нажмите отмену:",
    parse_mode="HTML",
    reply_markup=get_cancel_keyboard(),
)

@dp.message(UserState.waiting_system_prompt, F.text == "❌ Отмена")
async def cancel_system_prompt(message: Message, state: FSMContext):
    """Отмена изменения промпта."""
    await state.set_state(UserState.chatting)
    await message.answer(
        "❌ Изменение отменено.",
        reply_markup=get_main_keyboard(),
    )

@dp.message(UserState.waiting_system_prompt)
async def set_system_prompt(message: Message, state: FSMContext):
    """Установить новый системный промпт."""
    user_id = message.from_user.id
    new_prompt = message.text.strip()

    if len(new_prompt) < 5:
        await message.answer("⚠️ Промпт слишком короткий. Попробуйте ещё раз:")
        return

    user_system_prompts[user_id] = new_prompt
    clear_history(user_id) # Сбрасываем историю при смене промпта
    await state.set_state(UserState.chatting)

    await message.answer(
        f"✅ Системный промпт обновлён!\n\n"
        f"<i>{new_prompt}</i>\n\n"
        "История диалога сброшена. Можете начать новый разговор.",
        parse_mode="HTML",
        reply_markup=get_main_keyboard(),
    )

```

```

)

# =====
# Основной обработчик сообщений
# =====
@dp.message(UserState.chatting)
@dp.message(F.text & ~F.text.startswith("/"))
async def handle_message(message: Message, state: FSMContext):
    """Обработчик обычных сообщений — отправка в GPT."""
    user_id = message.from_user.id
    user_text = message.text.strip()

    if not user_text:
        await message.answer("🔇 Пожалуйста, введите текстовое сообщение.")
        return

    # Показываем индикатор печати
    await bot.send_chat_action(message.chat.id, "typing")

    try:
        system_prompt = get_system_prompt(user_id)
        response = await ask_gpt(user_id, user_text, system_prompt)

        # Разбиваем длинные сообщения
        if len(response) > 4096:
            for i in range(0, len(response), 4096):
                await message.answer(response[i : i + 4096])
        else:
            await message.answer(response)

    except Exception as e:
        logger.error(f"Ошибка для пользователя {user_id}: {e}")
        await message.answer(
            "❌ Произошла ошибка при обращении к ИИ.\n"
            "Попробуйте ещё раз или начните новый диалог.",
            reply_markup=get_main_keyboard(),
        )

# =====
# Запуск бота
# =====

```

```

async def set_commands():
    """Установить команды бота."""
    commands = [
        BotCommand(command="start", description="Запустить бота"),
        BotCommand(command="new", description="Начать новый диалог"),
        BotCommand(command="system", description="Изменить системный
промпт"),
        BotCommand(command="stats", description="Статистика диалога"),
        BotCommand(command="help", description="Помощь"),
    ]
    await bot.set_my_commands(commands)

async def main():
    """Основная функция запуска."""
    logger.info("Запуск бота...")
    await set_commands()
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())

```

Листинг 3.1 — config.py

```

import os
from dotenv import load_dotenv

load_dotenv()

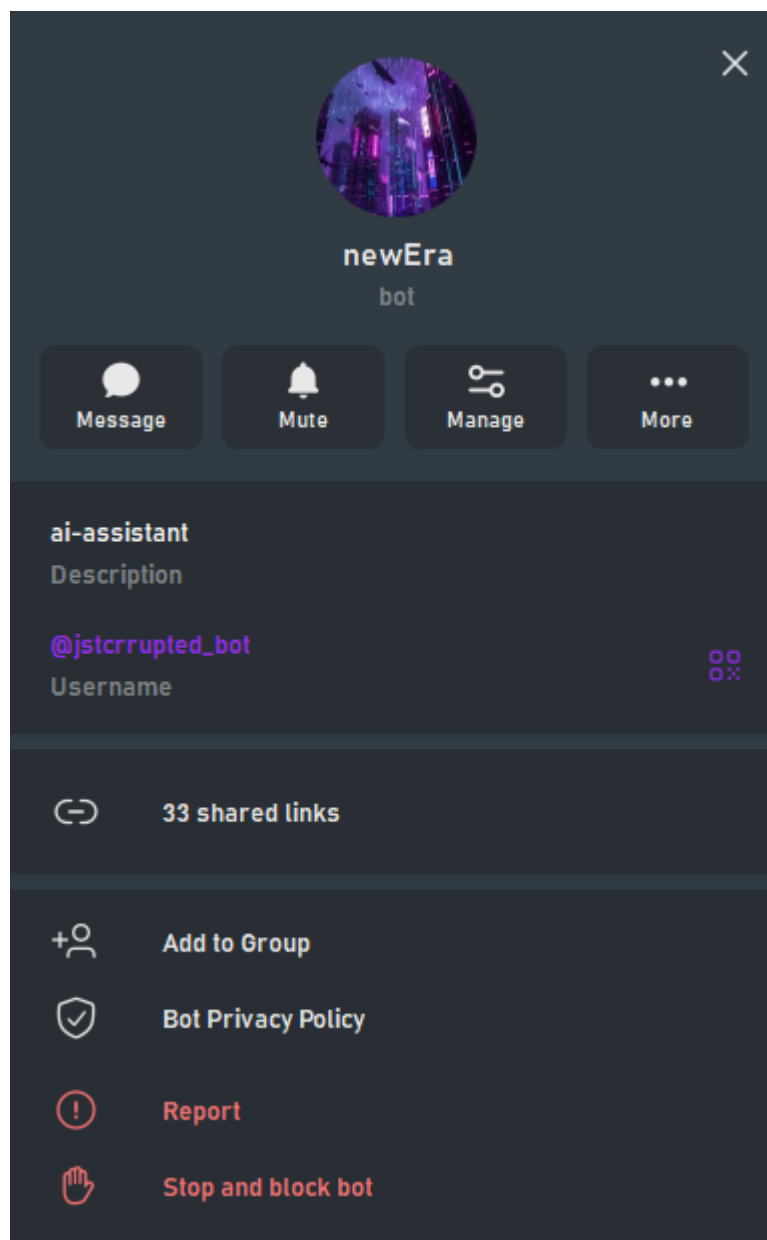
TELEGRAM_TOKEN = os.getenv("TELEGRAM_TOKEN")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

# Настройки GPT
GPT_MODEL = "gpt-3.5-turbo"
MAX_TOKENS = 1000
TEMPERATURE = 0.7

# Максимальная длина истории диалога
MAX_HISTORY_LENGTH = 10

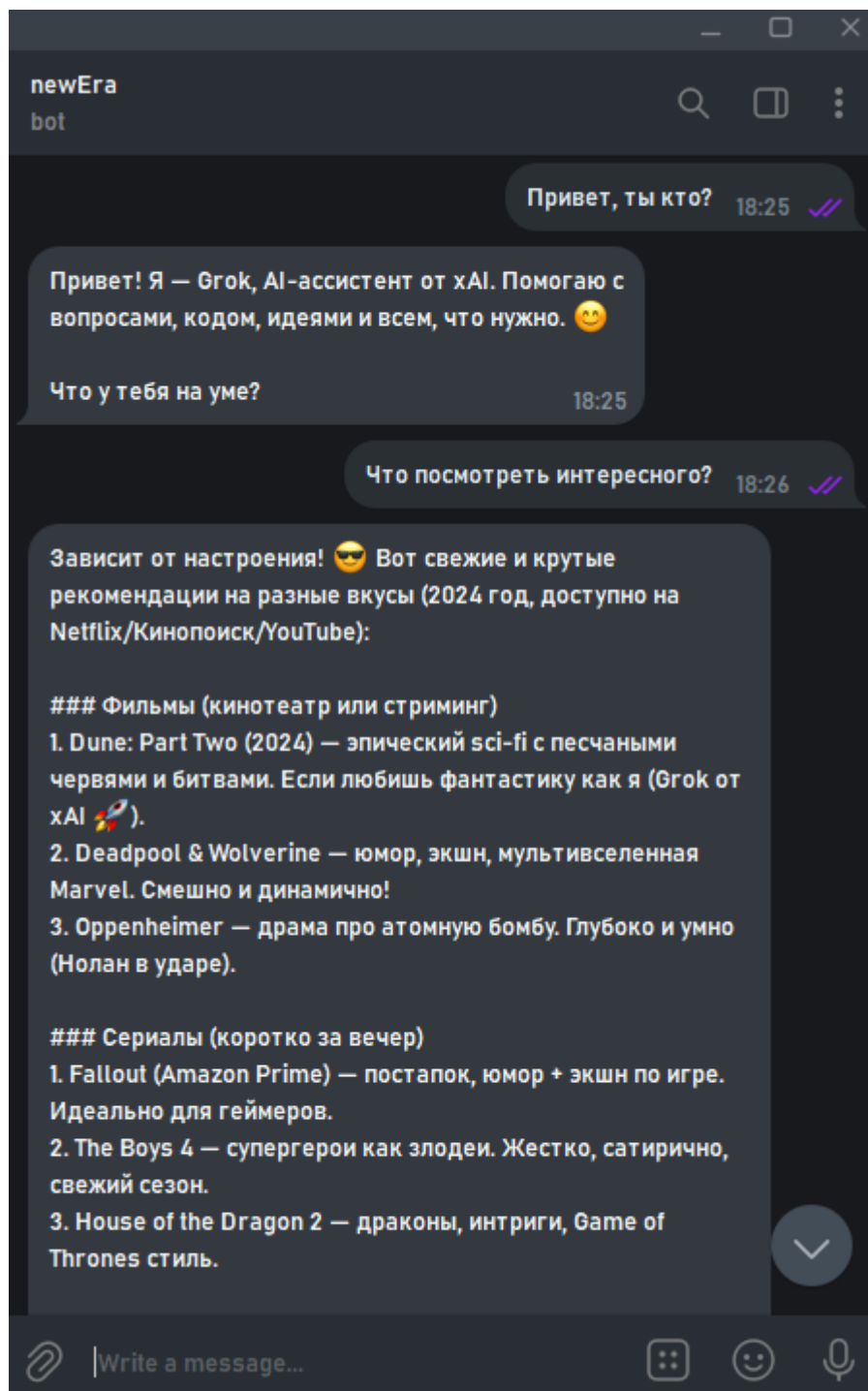
```

Скриншот И.1 — Описание тг-бота, аватар, юзернейм.



Скриншот И.1 — Описание бота

Скриншот К.1 — Работа тг-бота, ответы нейросети.



Скриншот К.1 — Работа бота